



Department of Computer Science and Engineering

R19IT251 – SOFTWARE ENGINEERING

MODULE I

1.1 SDLC

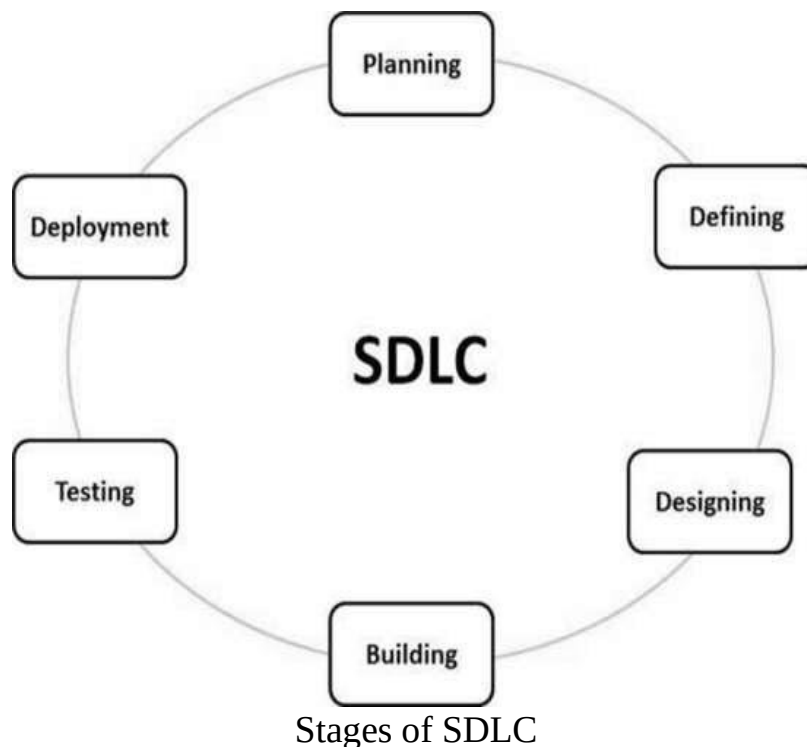
Software Development Life Cycle (SDLC) is a process used by the software industry to design, develop and test high quality softwares. The SDLC aims to produce a high-quality software that meets or exceeds customer expectations, reaches completion within times and cost estimates.

- SDLC is the acronym of Software Development Life Cycle.
- It is also called as Software Development Process.
- SDLC is a framework defining tasks performed at each step in the software development process.

What is SDLC?

SDLC is a process followed for a software project, within a software organization. It consists of a detailed plan describing how to develop, maintain, replace and alter or enhance specific software. The life cycle defines a methodology for improving the quality of software and the overall development process.

The following figure is a graphical representation of the various stages of a typical SDLC.



A typical Software Development Life Cycle consists of the following stages –

Stage 1: Planning and Requirement Analysis

Requirement analysis is the most important and fundamental stage in SDLC. It is performed by the senior members of the team with inputs from the customer, the sales department, market surveys and domain experts in the industry. This information is then used to plan the basic project approach and to conduct product feasibility study in the economical, operational and technical areas.

Planning for the quality assurance requirements and identification of the risks associated with the project is also done in the planning stage. The outcome of the technical feasibility study is to define the various technical approaches that can be followed to implement the project successfully with minimum risks.

Stage 2: Defining Requirements

Once the requirement analysis is done the next step is to clearly define and document the product requirements and get them approved from the customer or the market analysts. This is done through an SRS (Software Requirement Specification) document which consists of all the product requirements to be designed and developed during the project life cycle.

Stage 3: Designing the Product Architecture

SRS is the reference for product architects to come out with the best architecture for the product to be developed. Based on the requirements specified in SRS, usually more than one design approach for the product architecture is proposed and documented in a DDS - Design Document Specification.

This DDS is reviewed by all the important stakeholders and based on various parameters as risk assessment, product robustness, design modularity, budget and time constraints, the best design approach is selected for the product.

A design approach clearly defines all the architectural modules of the product along with its communication and data flow representation with the external and third party modules (if any). The internal design of all the modules of the proposed architecture should be clearly defined with the minutest of the details in DDS.

Stage 4: Building or Developing the Product

In this stage of SDLC the actual development starts and the product is built. The programming code is generated as per DDS during this stage. If the design is performed in a detailed and organized manner, code generation can be accomplished without much hassle.

Developers must follow the coding guidelines defined by their organization and programming tools like compilers, interpreters, debuggers, etc. are used to generate the code. Different high level programming languages such as C, C++, Pascal, Java

and PHP are used for coding. The programming language is chosen with respect to the type of software being developed.

Stage 5: Testing the Product

This stage is usually a subset of all the stages as in the modern SDLC models, the testing activities are mostly involved in all the stages of SDLC. However, this stage refers to the testing only stage of the product where product defects are reported, tracked, fixed and retested, until the product reaches the quality standards defined in the SRS.

Stage 6: Deployment in the Market and Maintenance

Once the product is tested and ready to be deployed it is released formally in the appropriate market. Sometimes product deployment happens in stages as per the business strategy of that organization. The product may first be released in a limited segment and tested in the real business environment (UAT- User acceptance testing).

Then based on the feedback, the product may be released as it is or with suggested enhancements in the targeting market segment. After the product is released in the market, its maintenance is done for the existing customer base.

1.2 SDLC Models

There are various software development life cycle models defined and designed which are followed during the software development process. These models are also referred as Software Development Process Models". Each process model follows a Series of steps unique to its type to ensure success in the process of software development.

Following are the most important and popular SDLC models followed in the industry —

- Waterfall Model
- Iterative Model
- Spiral Model
- V-Model
- Big Bang Model

Other related methodologies are Agile Model, RAD Model, Rapid Application Development and Prototyping Models.

1.2.1 SDLC - Waterfall Model

The Waterfall Model was the first Process Model to be introduced. It is also referred to as a linear-sequential life cycle model. It is very simple to understand and use. In a waterfall model, each phase must be completed before the next phase can begin and there is no overlapping in the phases.

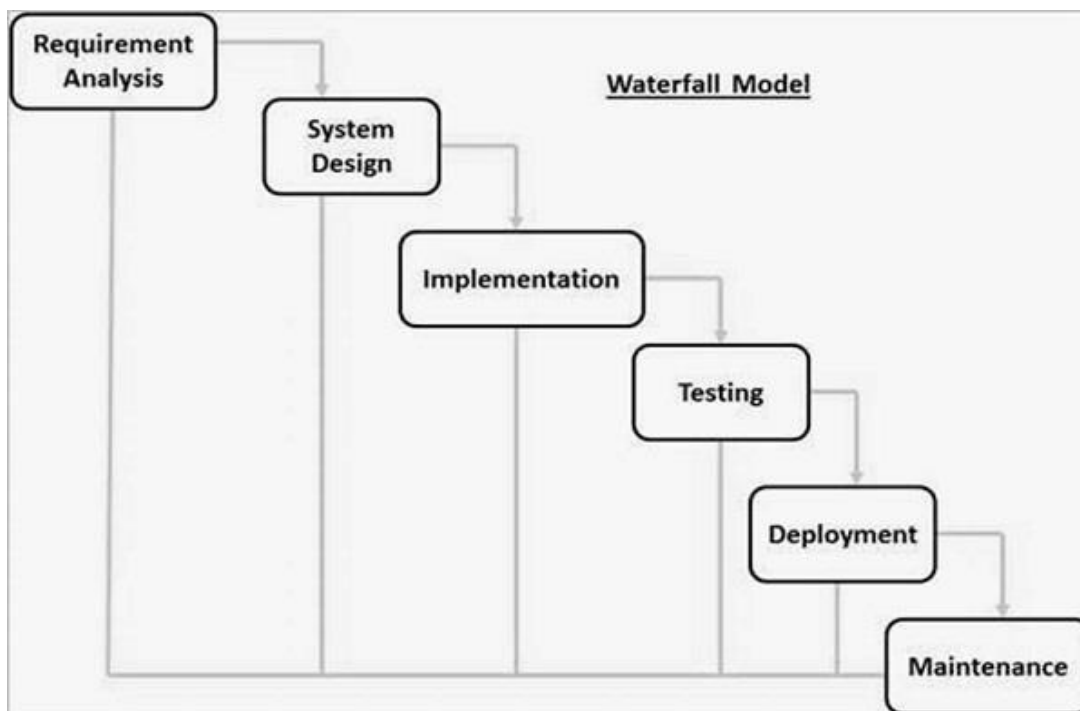
The Waterfall model is the earliest SDLC approach that was used for software development.

The waterfall Model illustrates the software development process in a linear sequential flow. This means that any phase in the development process begins only if the previous phase is complete. In this waterfall model, the phases do not overlap.

Waterfall Model - Design

Waterfall approach was first SDLC Model to be used widely in Software Engineering to ensure success of the project. In "The Waterfall" approach, the whole process of software development is divided into separate phases. In this Waterfall model, typically, the outcome of one phase acts as the input for the next phase sequentially.

The following illustration is a representation of the different phases of the Waterfall Model.



The sequential phases in Waterfall model are –

Requirement Gathering and analysis – All possible requirements of the system to be developed are captured in this phase and documented in a requirement specification document.

System Design – The requirement specifications from first phase are studied in this phase and the system design is prepared. This system design helps in specifying hardware and system requirements and helps in defining the overall system architecture.

Implementation – With inputs from the system design, the system is first developed in small programs called units, which are integrated in the next phase. Each unit is developed and tested for its functionality, which is referred to as Unit Testing.

Integration and Testing – All the units developed in the implementation phase are integrated into a system after testing of each unit. Post integration the entire system is tested for any faults and failures.

Deployment of system – Once the functional and non-functional testing is done; the product is deployed in the customer environment or released into the market.

Maintenance – There are some issues which come up in the client environment. To fix those issues, patches are released. Also to enhance the product some better versions are released. Maintenance is done to deliver these changes in the customer environment.

All these phases are cascaded to each other in which progress is seen as flowing steadily downwards (like a waterfall) through the phases. The next phase is started only after the defined set of goals are achieved for previous phase and it is signed off, so the name "Waterfall Model". In this model, phases do not overlap.

Waterfall Model - Application

Every software developed is different and requires a suitable SDLC approach to be followed based on the internal and external factors. Some situations where the use of Waterfall model is most appropriate are –

- Requirements are very well documented, clear and fixed.
- Product definition is stable.
- Technology is understood and is not dynamic.
- There are no ambiguous requirements.
- Ample resources with required expertise are available to support the product.
- The project is short.

Waterfall Model - Advantages

The advantages of waterfall development are that it allows for departmentalization and control. A schedule can be set with deadlines for each stage of development and a product can proceed through the development process model phases one by one.

Development moves from concept, through design, implementation, testing, installation, troubleshooting, and ends up at operation and maintenance. Each phase of development proceeds in strict order.

Some of the major advantages of the Waterfall Model are as follows –

- Simple and easy to understand and use
- Easy to manage due to the rigidity of the model. Each phase has specific deliverables and a review process.
- Phases are processed and completed one at a time.
- Works well for smaller projects where requirements are very well understood.
- Clearly defined stages.
- Well understood milestones.
- Easy to arrange tasks.
- Process and results are well documented.

Waterfall Model - Disadvantages

The disadvantage of waterfall development is that it does not allow much reflection or revision. Once an application is in the testing stage, it is very difficult to go back and change something that was not well-documented or thought upon in the concept stage.

The major disadvantages of the Waterfall Model are as follows –

- No working software is produced until late during the life cycle.
- High amounts of risk and uncertainty.
- Not a good model for complex and object-oriented projects.
- Poor model for long and ongoing projects.
- Not suitable for the projects where requirements are at a moderate to high risk of changing. So, risk and uncertainty is high with this process model.
- It is difficult to measure progress within stages.
- Cannot accommodate changing requirements.
- Adjusting scope during the life cycle can end a project.

Integration is done as a "big-bang. at the very end, which doesn't allow identifying any technological or business bottleneck or challenges early.

1.2.2 SDLC - Iterative Model

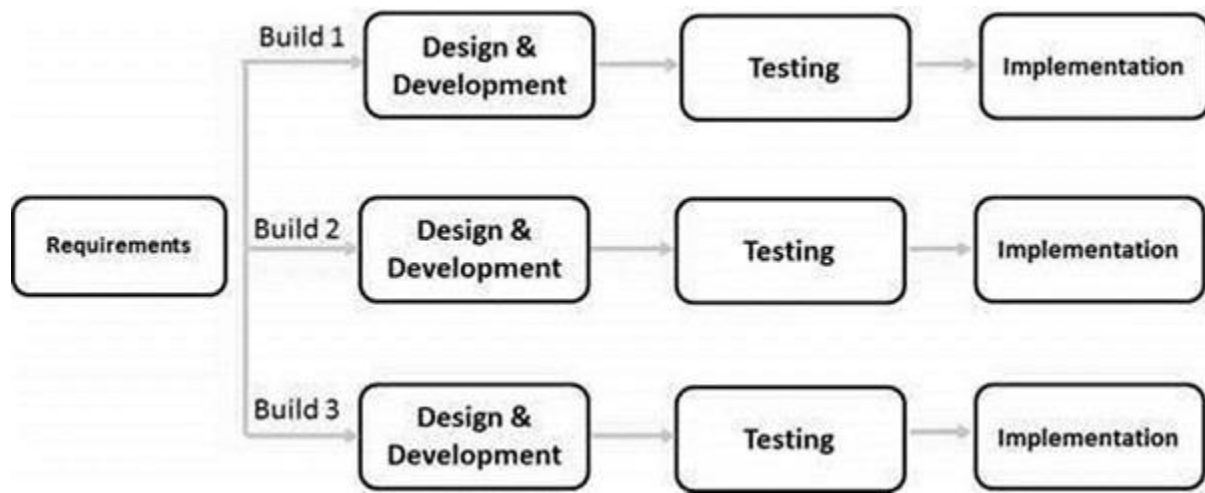
In the Iterative model, iterative process starts with a simple implementation of a small set of the software requirements and iteratively enhances the evolving versions until the complete system is implemented and ready to be deployed.

An iterative life cycle model does not attempt to start with a full specification of requirements. Instead, development begins by specifying and implementing just part of the software, which is then reviewed to identify further requirements. This process is then repeated, producing a new version of the software at the end of each iteration of the model.

Iterative Model - Design

Iterative process starts with a simple implementation of a subset of the software requirements and iteratively enhances the evolving versions until the full system is implemented. At each iteration, design modifications are made and new functional capabilities are added. The basic idea behind this method is to develop a system through repeated cycles (iterative) and in smaller portions at a time (incremental). The following illustration is a representation of the Iterative and Incremental model

—



- Iterative and Incremental development is a combination of both iterative design or iterative method and incremental build model for development. "During software development, more than one iteration of the software development cycle may be in progress at the same time." This process may be described as an "evolutionary acquisition" or "incremental build" approach."
- In this incremental model, the whole requirement is divided into various builds. During each iteration, the development module goes through the requirements, design, implementation and testing phases. Each subsequent release of the module adds function to the previous release. The process continues till the complete system is ready as per the requirement.
- The key to a successful use of an iterative software development lifecycle is rigorous validation of requirements, and verification & testing of each version of the software against those requirements within each cycle of the model. As the software evolves through successive cycles, tests must be repeated and extended to verify each version of the software.

Iterative Model – Application

Like other SDLC models, Iterative and incremental development has some specific applications in the software industry. This model is most often used in the following scenarios –

- Requirements of the complete system are clearly defined and understood.
- Major requirements must be defined; however, some functionalities or requested enhancements may evolve with time.
- There is a time to the market constraint.
- A new technology is being used and is being learnt by the development team while working on the project.
- Resources with needed skill sets are not available and are planned to be used on contract basis for specific iterations.
- There are some high-risk features and goals which may change in the future.

Iterative Model - Pros and Cons

The advantage of this model is that there is a working model of the system at a very early stage of development, which makes it easier to find functional or design flaws. Finding issues at an early stage of development enables to take corrective measures in a limited budget.

The disadvantage with this SDLC model is that it is applicable only to large and bulky software development projects. This is because it is hard to break a small software system into further small serviceable increments/modules.

The advantages of the Iterative and Incremental SDLC Model are as follows –

- Some working functionality can be developed quickly and early in the life cycle.
 - Results are obtained early and periodically.
 - Parallel development can be planned.
 - Progress can be measured.
 - Less costly to change the scope/requirements.
 - Testing and debugging during smaller iteration is easy.
 - Risks are identified and resolved during iteration; and each iteration is an easily managed milestone.
 - Easier to manage risk - High risk part is done first.
 - With every increment, operational product is delivered.
 - Issues, challenges and risks identified from each increment can be utilized/applied to the next increment.
 - Risk analysis is better.
 - It supports changing requirements.
 - Initial Operating time is less.
 - Better suited for large and mission-critical projects.
-
- During the life cycle, software is produced early which facilitates customer evaluation and feedback.

The disadvantages of the Iterative and Incremental SDLC Model are as follows –

- More resources may be required.
- Although cost of change is lesser, but it is not very suitable for changing requirements.
- More management attention is required.
- System architecture or design issues may arise because not all requirements are gathered in the beginning of the entire life cycle.
- Defining increments may require definition of the complete system.
- Not suitable for smaller projects.
- Management complexity is more.
- End of project may not be known which is a risk.
- Highly skilled resources are required for risk analysis.

- Projects progress is highly dependent upon the risk analysis phase.

1.2.3. SDLC - Spiral Model

The spiral model combines the idea of iterative development with the systematic, controlled aspects of the waterfall model. This Spiral model is a combination of iterative development process model and sequential linear development model i.e. the waterfall model with a very high emphasis on risk analysis. It allows incremental releases of the product or incremental refinement through each iteration around the spiral.

Spiral Model - Design

The spiral model has four phases. A software project repeatedly passes through these phases in iterations called Spirals.

Identification

This phase starts with gathering the business requirements in the baseline spiral. In the subsequent spirals as the product matures, identification of system requirements, subsystem requirements and unit requirements are all done in this phase.

This phase also includes understanding the system requirements by continuous communication between the customer and the system analyst. At the end of the spiral, the product is deployed in the identified market.

Design

The Design phase starts with the conceptual design in the baseline spiral and involves architectural design, logical design of modules, physical product design and the final design in the subsequent spirals.

Construct or Build

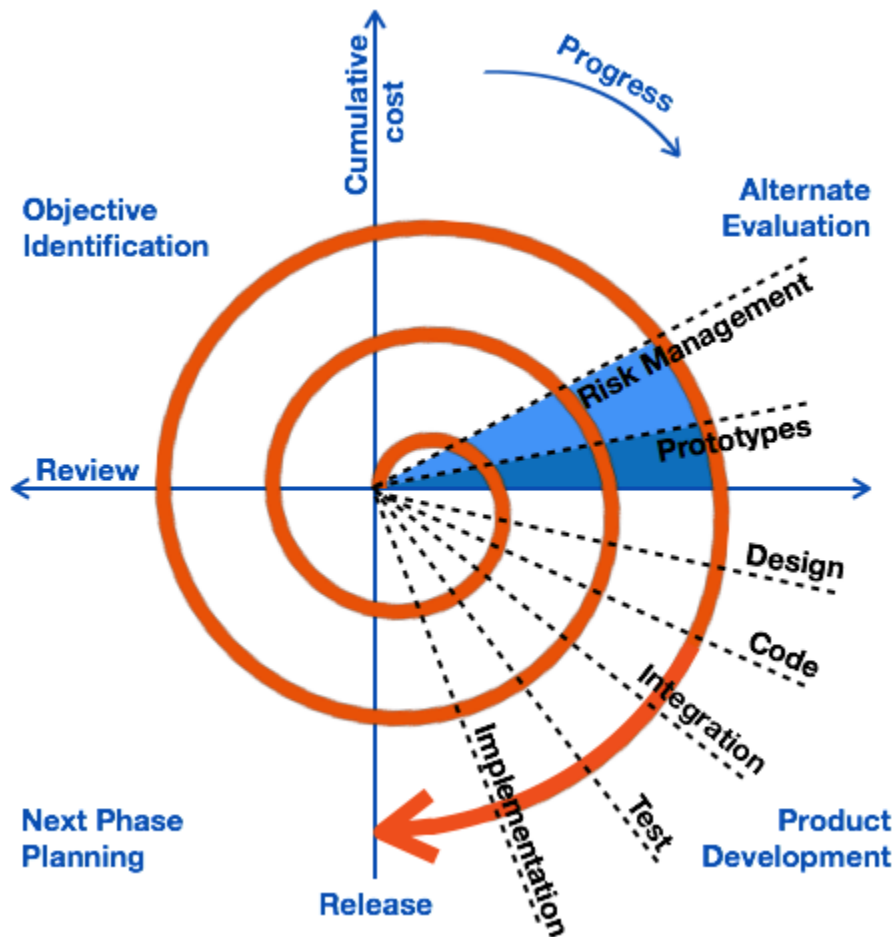
The Construct phase refers to production of the actual software product at every spiral. In the baseline spiral, when the product is just thought of and the design is being developed a POC (Proof of Concept) is developed in this phase to get customer feedback.

Then in the subsequent spirals with higher clarity on requirements and design details a working model of the software called build is produced with a version number. These builds are sent to the customer for feedback.

Evaluation and Risk Analysis

Risk Analysis includes identifying, estimating and monitoring the technical feasibility and management risks, such as schedule slippage and cost overrun. After testing the build, at the end of first iteration, the customer evaluates the software and provides feedback.

The following illustration is a representation of the Spiral Model, listing the activities in each phase.



Based on the customer evaluation, the software development process enters the next iteration and subsequently follows the linear approach to implement the feedback suggested by the customer. The process of iterations along the spiral continues throughout the life of the software.

Spiral Model Application

The Spiral Model is widely used in the software industry as it is in sync with the natural development process of any product, i.e. learning with maturity which involves minimum risk for the customer as well as the development firms.

The following pointers explain the typical uses of a Spiral Model –

- When there is a budget constraint and risk evaluation is important.
- For medium to high-risk projects.
- Long-term project commitment because of potential changes to economic priorities as the requirements change with time.
- Customer is not sure of their requirements which is usually the case.
- Requirements are complex and need evaluation to get clarity.
- New product line which should be released in phases to get enough customer feedback.
- Significant changes are expected in the product during the development cycle.

Spiral Model - Pros and Cons

- The advantage of spiral lifecycle model is that it allows elements of the product to be added in, when they become available or known. This assures that there is no conflict with previous requirements and design.
- This method is consistent with approaches that have multiple software builds and releases which allows making an orderly transition to a maintenance activity. Another positive aspect of this method is that the spiral model forces an early user involvement in the system development effort.
- On the other side, it takes a very strict management to complete such products and there is a risk of running the spiral in an indefinite loop. So, the discipline of change and the extent of taking change requests is very important to develop and deploy the product successfully.

The advantages of the Spiral SDLC Model are as follows –

- Changing requirements can be accommodated.
- Allows extensive use of prototypes.
- Requirements can be captured more accurately.
- Users see the system early.
- Development can be divided into smaller parts and the risky parts can be developed earlier which helps in better risk management.

The disadvantages of the Spiral SDLC Model are as follows –

- Management is more complex.
- End of the project may not be known early.
- Not suitable for small or low risk projects and could be expensive for small projects.
- Process is complex
- Spiral may go on indefinitely.
- Large number of intermediate stages requires excessive documentation.

1.2.4 SDLC - V-Model

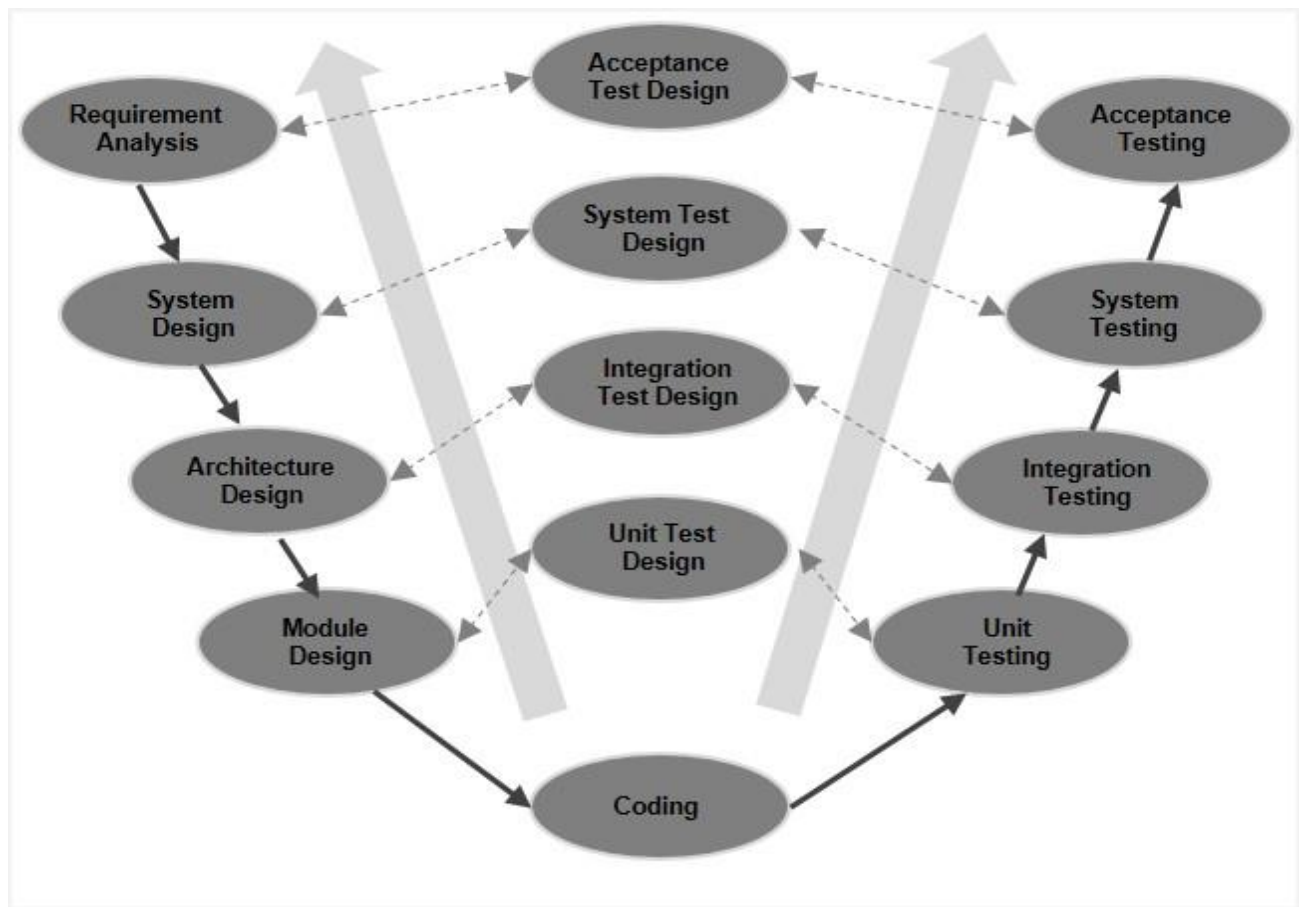
The V-model is an SDLC model where execution of processes happens in a sequential manner in a V-shape. It is also known as Verification and Validation model.

The V-Model is an extension of the waterfall model and is based on the association of a testing phase for each corresponding development stage. This means that for every single phase in the development cycle, there is a directly associated testing phase. This is a highly-disciplined model and the next phase starts only after completion of the previous phase.

V-Model - Design

Under the V-Model, the corresponding testing phase of the development phase is planned in parallel. So, there are Verification phases on one side of the 'V' and Validation phases on the other side. The Coding Phase joins the two sides of the V-Model.

The following illustration depicts the different phases in a V-Model of the SDLC.



V-Model - Verification Phases

There are several Verification phases in the V-Model, each of these are explained in detail below.

Business Requirement Analysis

This is the first phase in the development cycle where the product requirements are understood from the customer's perspective. This phase involves detailed communication with the customer to understand his expectations and exact requirement. This is a very important activity and needs to be managed well, as most of the customers are not sure about what exactly they need. The acceptance test design planning is done at this stage as business requirements can be used as an input for acceptance testing.

System Design

Once you have the clear and detailed product requirements, it is time to design the complete system. The system design will have the understanding and detailing the complete hardware and communication setup for the product under development. The system test plan is developed based on the system design. Doing this at an earlier stage leaves more time for the actual test execution later.

Architectural Design

Architectural specifications are understood and designed in this phase. Usually more than one technical approach is proposed and based on the technical and financial feasibility the final decision is taken. The system design is broken down further into modules taking up different functionality. This is also referred to as High Level Design (HLD).

The data transfer and communication between the internal modules and with the outside world (other systems) is clearly understood and defined in this stage. With this information, integration tests can be designed and documented during this stage.

Module Design

In this phase, the detailed internal design for all the system modules is specified, referred to as Low Level Design (LLD). It is important that the design is compatible with the other modules in the system architecture and the other external systems. The unit tests are an essential part of any development process and helps eliminate the maximum faults and errors at a very early stage. These unit tests can be designed at this stage based on the internal module designs.

Coding Phase

The actual coding of the system modules designed in the design phase is taken up in the Coding phase. The best suitable programming language is decided based on the system and architectural requirements.

The coding is performed based on the coding guidelines and standards. The code goes through numerous code reviews and is optimized for best performance before the final build is checked into the repository.

Validation Phases

The different Validation Phases in a V-Model are explained in detail below.

Unit Testing

Unit tests designed in the module design phase are executed on the code during this validation phase. Unit testing is the testing at code level and helps eliminate bugs at an early stage, though all defects cannot be uncovered by unit testing.

Integration Testing

Integration testing is associated with the architectural design phase. Integration tests are performed to test the coexistence and communication of the internal modules within the system.

System Testing

System testing is directly associated with the system design phase. System tests check the entire system functionality and the communication of the system under development with external systems. Most of the software and hardware compatibility issues can be uncovered during this system test execution.

Acceptance Testing

Acceptance testing is associated with the business requirement analysis phase and involves testing the product in user environment. Acceptance tests uncover the compatibility issues with the other systems available in the user environment. It also discovers the non-functional issues such as load and performance defects in the actual user environment.

V- Model — Application

V- Model application is almost the same as the waterfall model, as both the models are of sequential type. Requirements have to be very clear before the project starts, because it is usually expensive to go back and make changes. This model is used in the medical development field, as it is strictly a disciplined domain.

The following pointers are some of the most suitable scenarios to use the V-Model application.

- Requirements are well defined, clearly documented and fixed.
- Product definition is stable.
- Technology is not dynamic and is well understood by the project team.
- There are no ambiguous or undefined requirements.
- The project is short.

V-Model - Pros and Cons

The advantage of the V-Model method is that it is very easy to understand and apply. The simplicity of this model also makes it easier to manage. The disadvantage is that the model is not flexible to changes and just in case there is a requirement change, which is very common in today's dynamic world, it becomes very expensive to make the change.

The advantages of the V-Model method are as follows –

- This is a highly-disciplined model and Phases are completed one at a time.
- Works well for smaller projects where requirements are very well understood.
- Simple and easy to understand and use.
- Easy to manage due to the rigidity of the model. Each phase has specific deliverables and a review process.

The disadvantages of the V-Model method are as follows –

- High risk and uncertainty.
- Not a good model for complex and object-oriented projects.
- Poor model for long and ongoing projects.
- Not suitable for the projects where requirements are at a moderate to high risk of changing.
- Once an application is in the testing stage, it is difficult to go back and change a functionality.
- No working software is produced until late during the life cycle.

1.2.5. SDLC - Big Bang Model

The Big Bang model is an SDLC model where we do not follow any specific process. The development just starts with the required money and efforts as the input, and the output is the software developed which may or may not be as per customer requirement. This Big Bang Model does not follow a process/procedure and there is a very little planning required. Even the customer is not sure about what exactly he wants and the requirements are implemented on the fly without much analysis.

Usually this model is followed for small projects where the development teams are very small.

Big Bang Model – Design and Application

The Big Bang Model comprises of focusing all the possible resources in the software development and coding, with very little or no planning. The requirements are

understood and implemented as they come. Any changes required may or may not need to revamp the complete software.

This model is ideal for small projects with one or two developers working together and is also useful for academic or practice projects. It is an ideal model for the product where requirements are not well understood and the final release date is not given.

Big Bang Model - Pros and Cons

The advantage of this Big Bang Model is that it is very simple and requires very little or no planning. Easy to manage and no formal procedure are required.

However, the Big Bang Model is a very high risk model and changes in the requirements or misunderstood requirements may even lead to complete reversal or scraping of the project. It is ideal for repetitive or small projects with minimum risks.

The advantages of the Big Bang Model are as follows –

- This is a very simple model
- Little or no planning required
- Easy to manage
- Very few resources required
- Gives flexibility to developers
- It is a good learning aid for new comers or students.

The disadvantages of the Big Bang Model are as follows –

- Very High risk and uncertainty.
- Not a good model for complex and object-oriented projects.
- Poor model for long and ongoing projects.
- Can turn out to be very expensive if requirements are misunderstood.

1.2.6. SDLC - Agile Model

Agile SDLC model is a combination of iterative and incremental process models with focus on process adaptability and customer satisfaction by rapid delivery of working software product. Agile Methods break the product into small incremental builds. These builds are provided in iterations. Each iteration typically lasts from about one to three weeks. Every iteration involves cross functional teams working simultaneously on various areas like –

- Planning
- Requirements Analysis
- Design
- Coding
- Unit Testing and
- Acceptance Testing.

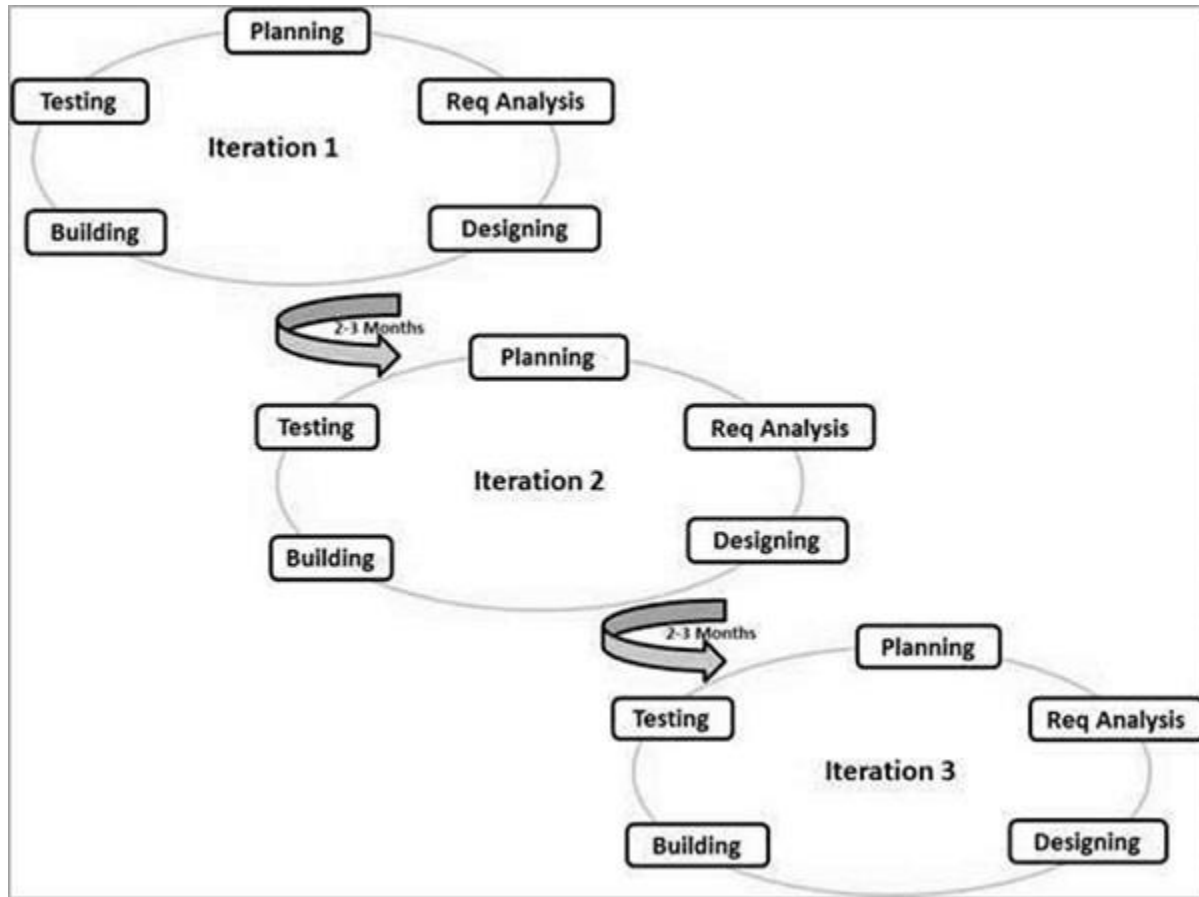
At the end of the iteration, a working product is displayed to the customer and important stakeholders.

What is Agile?

Agile model believes that every project needs to be handled differently and the existing methods need to be tailored to best suit the project requirements. In Agile, the tasks are divided to time boxes (small time frames) to deliver specific features for a release.

Iterative approach is taken and working software build is delivered after each iteration. Each build is incremental in terms of features; the final build holds all the features required by the customer.

Here is a graphical illustration of the Agile Model –



The Agile thought process had started early in the software development and started becoming popular with time due to its flexibility and adaptability.

The most popular Agile methods include Rational Unified Process (1994), Scrum (1995), Crystal Clear, Extreme Programming (1996), Adaptive Software Development, Feature Driven Development, and Dynamic Systems Development Method (DSDM) (1995). These are now collectively referred to as Agile Methodologies, after the Agile Manifesto was published in 2001.

Following are the Agile Manifesto principles –

- **Individuals and interactions** – In Agile development, self-organization and motivation are important, as are interactions like co-location and pair programming.

- **Working software** – Demo working software is considered the best means of communication with the customers to understand their requirements, instead of just depending on documentation.
- **Customer collaboration** – As the requirements cannot be gathered completely in the beginning of the project due to various factors, continuous customer interaction is very important to get proper product requirements.
- **Responding to change** – Agile Development is focused on quick responses to change and continuous development.

Agile Vs Traditional SDLC Models

Agile is based on the adaptive software development methods, whereas the traditional SDLC models like the waterfall model is based on a predictive approach. Predictive teams in the traditional SDLC models usually work with detailed planning and have a complete forecast of the exact tasks and features to be delivered in the next few months or during the product life cycle.

Predictive methods entirely depend on the requirement analysis and planning done in the beginning of cycle. Any changes to be incorporated go through a strict change control management and prioritization.

Agile uses an adaptive approach where there is no detailed planning and there is clarity on future tasks only in respect of what features need to be developed. There is feature driven development and the team adapts to the changing product requirements dynamically. The product is tested very frequently, through the release iterations, minimizing the risk of any major failures in future.

Customer Interaction is the backbone of this Agile methodology, and open communication with minimum documentation are the typical features of Agile development environment. The agile teams work in close collaboration with each other and are most often located in the same geographical location.

Agile Model - Pros and Cons

Agile methods are being widely accepted in the software world recently. However, this method may not always be suitable for all products. Here are some pros and cons of the Agile model.

The advantages of the Agile Model are as follows –

- Is a very realistic approach to software development.
- Promotes teamwork and cross training.
- Functionality can be developed rapidly and demonstrated.
- Resource requirements are minimum.
- Suitable for fixed or changing requirements
- Delivers early partial working solutions.
- Good model for environments that change steadily.
- Minimal rules, documentation easily employed.

- Enables concurrent development and delivery within an overall planned context.
- Little or no planning required.
- Easy to manage.
- Gives flexibility to developers.

The disadvantages of the Agile Model are as follows –

- Not suitable for handling complex dependencies.
- More risk of sustainability, maintainability and extensibility.
- An overall plan, an agile leader and agile PM practice is a must without which it will not work.
- Strict delivery management dictates the scope, functionality to be delivered, and adjustments to meet the deadlines.
- Depends heavily on customer interaction, so if customer is not clear, team can be driven in the wrong direction.
- There is a very high individual dependency, since there is minimum documentation generated.
- Transfer of technology to new team members may be quite challenging due to lack of documentation.

1.2.7 SDLC - Software Prototype Model

The Software Prototyping refers to building software application prototypes which displays the functionality of the product under development, but may not actually hold the exact logic of the original software.

Software prototyping is becoming very popular as a software development model, as it enables to understand customer requirements at an early stage of development. It helps get valuable feedback from the customer and helps software designers and developers understand about what exactly is expected from the product under development.

What is Software Prototyping?

Prototype is a working model of software with some limited functionality. The prototype does not always hold the exact logic used in the actual software application and is an extra effort to be considered under effort estimation.

Prototyping is used to allow the users evaluate developer proposals and try them out before implementation. It also helps understand the requirements which are user specific and may not have been considered by the developer during product design.

Following is a stepwise approach explained to design a software prototype.

Basic Requirement Identification

This step involves understanding the very basics product requirements especially in terms of user interface. The more intricate details of the internal design and external aspects like performance and security can be ignored at this stage.

Developing the initial Prototype

The initial Prototype is developed in this stage, where the very basic requirements are showcased and user interfaces are provided. These features may not exactly work in the same manner internally in the actual software developed. While, the workarounds are used to give the same look and feel to the customer in the prototype developed.

Review of the Prototype

The prototype developed is then presented to the customer and the other important stakeholders in the project. The feedback is collected in an organized manner and used for further enhancements in the product under development.

Revise and Enhance the Prototype

The feedback and the review comments are discussed during this stage and some negotiations happen with the customer based on factors like – time and budget constraints and technical feasibility of the actual implementation. The changes accepted are again incorporated in the new Prototype developed and the cycle repeats until the customer expectations are met.

Prototypes can have horizontal or vertical dimensions. A Horizontal prototype displays the user interface for the product and gives a broader view of the entire system, without concentrating on internal functions. A Vertical prototype on the other side is a detailed elaboration of a specific function or a sub system in the product.

The purpose of both horizontal and vertical prototype is different. Horizontal prototypes are used to get more information on the user interface level and the business requirements. It can even be presented in the sales demos to get business in the market. Vertical prototypes are technical in nature and are used to get details of the exact functioning of the sub systems. For example, database requirements, interaction and data processing loads in a given sub system.

Software Prototyping - Types

There are different types of software prototypes used in the industry. Following are the major software prototyping types used widely –

Throwaway/Rapid Prototyping

Throwaway prototyping is also called as rapid or close ended prototyping. This type of prototyping uses very little efforts with minimum requirement analysis to build a prototype. Once the actual requirements are understood, the prototype is discarded

and the actual system is developed with a much clear understanding of user requirements.

Evolutionary Prototyping

Evolutionary prototyping also called as breadboard prototyping is based on building actual functional prototypes with minimal functionality in the beginning. The prototype developed forms the heart of the future prototypes on top of which the entire system is built. By using evolutionary prototyping, the well-understood requirements are included in the prototype and the requirements are added as and when they are understood.

Incremental Prototyping

Incremental prototyping refers to building multiple functional prototypes of the various sub-systems and then integrating all the available prototypes to form a complete system.

Extreme Prototyping

Extreme prototyping is used in the web development domain. It consists of three sequential phases. First, a basic prototype with all the existing pages is presented in the HTML format. Then the data processing is simulated using a prototype services layer. Finally, the services are implemented and integrated to the final prototype. This process is called Extreme Prototyping used to draw attention to the second phase of the process, where a fully functional UI is developed with very little regard to the actual services.

Software Prototyping - Application

Software Prototyping is most useful in development of systems having high level of user interactions such as online systems. Systems which need users to fill out forms or go through various screens before data is processed can use prototyping very effectively to give the exact look and feel even before the actual software is developed.

Software that involves too much of data processing and most of the functionality is internal with very little user interface does not usually benefit from prototyping. Prototype development could be an extra overhead in such projects and may need lot of extra efforts.

Software Prototyping - Pros and Cons

Software prototyping is used in typical cases and the decision should be taken very carefully so that the efforts spent in building the prototype add considerable value to the final software developed. The model has its own pros and cons discussed as follows.

The advantages of the Prototyping Model are as follows –

- Increased user involvement in the product even before its implementation.

- Since a working model of the system is displayed, the users get a better understanding of the system being developed.
- Reduces time and cost as the defects can be detected much earlier.
- Quicker user feedback is available leading to better solutions.
- Missing functionality can be identified easily.
- Confusing or difficult functions can be identified.

The Disadvantages of the Prototyping Model are as follows –

- Risk of insufficient requirement analysis owing to too much dependency on the prototype.
- Users may get confused in the prototypes and actual systems.
- Practically, this methodology may increase the complexity of the system as scope of the system may expand beyond original plans.
- Developers may try to reuse the existing prototypes to build the actual system, even when it is not technically feasible.
- The effort invested in building prototypes may be too much if it is not monitored properly.

1.3 Agile Scrum Framework: Values, Roles, Events, Artifacts

Scrum has been the most popular Agile methodology among enterprises for many years, and for good reasons. For example, by adopting Scrum Agile methodology, an organization can enjoy multiple benefits such as improved product quality, increased flexibility, time and money savings, better resource utilization, and more.

However, to reap all the benefits of the Agile Scrum process, the organization must be aware of the Scrum principles, Scrum framework, and all its nuances and facets. This blog will provide detailed insights into each of the key elements of the Agile Scrum framework to help you understand the concept.

What is Scrum Methodology?

Scrum in Agile is a project management methodology that focuses on iterative and incremental development. In this project development approach, the project is divided into multiple parts called sprints. Each iteration typically consists of two to four sprints.

In contrast to other project development methodologies, Scrum in Agile focuses on building and delivering features in batches. The features are selected based on their value and importance to customers. This approach provides **multiple advantages** to the organization, including:

- Faster product development
- Improvement of focus on the sprint's ongoing objectives
- Greater room for creativity and innovation
- Enhanced adaptability and flexibility
- Keeping the workforce motivated
- Enhanced customer satisfaction

- However, there are several elements in the Scrum framework that contribute to Scrum being an effective project development Agile methodology. In other words, Scrum is a framework within which the Scrum team addresses complex project-related problems.

1.3.1 The Agile Scrum Framework

Scrum is a lightweight framework that is more than just a methodology. It employs the scientific method of empiricism, which replaces the programmed, algorithmic approach with one based on heuristics, allowing organizations and individuals to deal with complex and unpredictable problems with ease.

1.3.2 The Agile Scrum framework consists of several components, including:

1. Scrum values
2. Scrum roles
3. Scrum events
4. Scrum artifacts

Again, each of these components consists of different elements, as explained below. Let us go over each one briefly.

1.3.2.1 Scrum Values

Commitment, Openness, Focus, Respect, and Courage are the five Scrum values. To achieve Scrum project management success, each Scrum team member must adhere to these values.

A. Commitment

As the name implies, one of the key Scrum values essential in the success of any Scrum project management is commitment. All teams and team members must adhere to the Scrum principles and collaborate as a unit to achieve common goals.

B. Focus

Focus is another Scrum model value that is important in ensuring that each sprint is completed on time and with the highest possible quality of work. To accomplish this objective, each team and Scrum team member must be more focused on their responsibilities and objectives.

C. Openness

Transparency or openness among Scrum teams and team members is crucial for Scrum project management or for the Agile Scrum process. Each of the project's teams and stakeholders is expected to be open and honest about their progress and challenges in completing tasks.

D. Respect

Each member of the Scrum team is expected to respect one another. When it comes to respect, an individual's role or position should not come into play. Respect for each individual will eventually result in a positive and healthy workplace where everyone will be able to give their best.

E. Courage

Scrum team members should have the belief and confidence to take the right step and do the right thing, and work on problems that are complex in nature. This is only possible if they are open about their challenges and work, as well as transparent, focused, and committed to their work.

1.3.2.2 Scrum Roles

The Scrum team primarily consists of three roles: Product Owner, Scrum Master, and Scrum Team. Each of these roles has a specific set of responsibilities to fulfill.

A. Product Owner

Product Owner, also known as PO, is an individual, who is in charge of ensuring that products are delivered on time and with the best quality. The Product Owner is also in charge of maximizing the product's value and the work of the development team.

The following are some of the primary duties of a Product Owner:

- Accountable for the product's success
- Product backlog management and prioritization
- Maximizing value to product quality
- Stakeholder management
- Direct the project in the right direction
- Defining and announcing releases

B. Scrum Master

The Scrum Master is a position that is responsible for providing all guidance, resources, and support to the Scrum team in order to help them work at their best. It is their responsibility to create an environment for the Scrum team in which everyone can work without distraction.

Some of the primary responsibilities of a Scrum Master include:

- Keeping the team focused, organized, and motivated
- Facilitating Scrum events like daily stand-ups and meetings as needed
- Making sure that everyone in the team understands the scope and goal of the project
- Conducting a retrospective sprint
- Helping in grooming the product backlog

C. Scrum Team

A Scrum team, also known as a development team, is a group of people who work together to complete and deliver the requested product, feature, or other requirements. The Scrum team prioritizes items from the product backlog based on the Scrum master's guidance.

Some of the Scrum team's key responsibilities include:

- On-time completion of the sprint work
- In charge of managing the product backlog
- To maintain transparency throughout the sprint
- To inspect and adapt the work on a regular basis
- Use data to identify the best development practices
- Test prototypes and products

- Remember that each of these Scrum roles is critical to the successful implementation of the Agile Scrum process.

1.3.2.3 Scrum Events



The Scrum events, also known as Scrum ceremonies, are the activities that take place during the sprint. The four Scrum events are Sprint Planning, Daily Scrum Meeting, Sprint Review, and Sprint Retrospective. Let's take a brief look at each one.

A. Sprint Planning

Sprint planning is the first event in the Scrum model where the entire sprint's planning is outlined and discussed. The goal of this event is to plan out how the work will be delivered, which component will be developed first, and how the project will be executed. The entire team, including the Scrum master, Scrum product manager, and Scrum team, usually attends this event.

B. Daily Scrum Meeting

Daily scrum meeting is a 15 to 20-minute stand-up meeting held every day to track sprint progress. Every member of the sprint team must attend the Scrum meeting. During the Scrum stand-up meeting, the relevant authority, typically the Scrum master or the product manager, discusses and addresses any difficulties experienced by an individual or a team.

C. Sprint Review

Sprint review is a collaborative session that happens at the end of the sprint. This Scrum event aims to review how well the sprint has been completed, what challenges the team experienced during the sprint, and what needs to be addressed in the next sprint. Every team and the members who were a part of the sprint, including the product owner, product manager, scrum master, and stakeholders, usually attend the sprint review meeting.

D. Sprint Retrospective

Sprint retrospective is a recurring meeting held at the end of a sprint to discuss the challenges and issues that the team encountered during the sprint. Unlike the Sprint review, which focuses on the product, the Sprint retrospective focuses on the team. The goal of this Scrum event is to increase team efficiency, happiness, and focus. Sprint retrospective meetings are typically attended by the Scrum team, which includes the Scrum master and the product owner.

1.3.2.4 Scrum Artifacts

Scrum artifacts are meta data that the Scrum team uses to improve their understanding of the product being developed as well as the Agile Scrum process. Scrum artifacts also provide information about the Scrum project management activities and product scope.

A. Product Backlog

A product backlog is a list of items or works that need to be worked on or delivered. The items on the product backlog are prioritized based on their importance, value, and timeframe. A well-organized product backlog not only helps teams plan and perform better but also assists stakeholders and other teams in setting expectations. Furthermore, a well-defined and prioritized backlog simplifies release and iteration planning.

B. Sprint Backlog

The sprint backlog is a collection of items or tasks that your team is expected to complete during the sprint. These tasks are typically pulled from the product backlog during the sprint planning event that is held at the start of the Agile Scrum process. A well-monitored and prioritized sprint backlog enables the team to understand the scope and requirements of the product and complete the work on time and with the highest standard.

C. Sprint Goal

A sprint goal is a third important Scrum artifact that describes what the team expects to accomplish at the end of the sprint. These objectives are usually discussed and written down during the sprint planning event. Everyone from the Scrum team to the scrum master to the product owner provides feedback when deciding on sprint goals. Again, sprint goals are divided into multiple sub-goals to help better organize the Agile Scrum process.

1.3.2.5 Scrum History

The Agile Scrum process is now a very popular software development concept. However, you might be surprised to learn that it did not even exist or was discovered before three decades!

Let's take a quick look at the origins, growth, and evolution of Scrum project management.

A. Origin

In 1986, authors Hirotaka Takeuchi and Ikujiro Nonaka coined the term Scrum in the context of product development. The authors introduced a holistic or rugby approach to commercial product development in their article, *The New New Product Development Game* (1986). Later, at the DuPont Research Station and the University of Dalware, Ken Schwaber and Babatunde introduced the Scrum framework.

In the early 1990s, Ken Schwaber's company, Advanced Development Methods, used what would become Scrum. On the other hand, at Easel Corporation, the trio of Jeff Sutherland, John Scumniotales, and Jeff McKenna developed a similar product development approach known as Scrum.

B. Scrum & Software Development

Sutherland and Schwaber continued to work on improving the Scrum framework over time, leveraging their experience and evolving best practices. They presented a Scrum paper in 1995, contributed to the Manifesto for Agile Software Development in 2001, and have been involved in the global spread and use of Scrum since 2002. Sutherland, Schwaber, and a community of Scrum practitioners have since begun implementing the Scrum framework in a variety of organizations, in addition to assisting them in the formation of high-performing teams. In the same year, 2002, Ken Schwaber co-founded Scrum Alliance with Mike Cohn and Esther Derby. They also launched the Certified Scrum Master (SCM) program in the years that followed.

C. Scrum in the 2000s

In 2014, Dr Dave Cornelius, a globally recognized Lean and Agile catalyst, presented his doctoral research on the Scrum value. During this time, Scrum continued to evolve. After two years, in 2016, they recognized the need for a Scrum framework for distributed teams and two product owners. That's how the first fully scalable Scrum was formed.

Since then, millions of businesses have started using Scrum for software development. It is not an exaggeration to say that Scrum has forever changed the software development landscape and made it far superior to traditional methodologies.

Conclusion

Scrum in Agile methodology has significantly changed the software development landscape over the last decade. It has made the development process more value-driven, streamlined, and efficient in terms of time, resources, and money. However, in order to reap all the great benefits of Scrum project management, organizations must first understand the Scrum framework and its nuances. A better understanding of the Agile Scrum framework and its components enables them to implement Scrum more effectively and achieve better results.

What is Scrum?

Scrum is a simple empirical process that enables teams to build products incrementally in iterations, to keep abreast of the changing market needs and align themselves to the organization's business goals.

"Scrum is an agile process that allows us to focus on delivering the highest business value in the shortest time."

- Mike Cohn, Mountain Goat Software

Scrum advocates self-organizing teams working towards a common goal through continuous inspection and adaptation. A minimum viable product at the end of each iteration provides an option for the teams to quickly get feedback from end users and respond much faster.

Scrum Methodology

Scrum is simple light weighted agile project management methodology that enables product teams to build products incrementally in an iterative fashion through

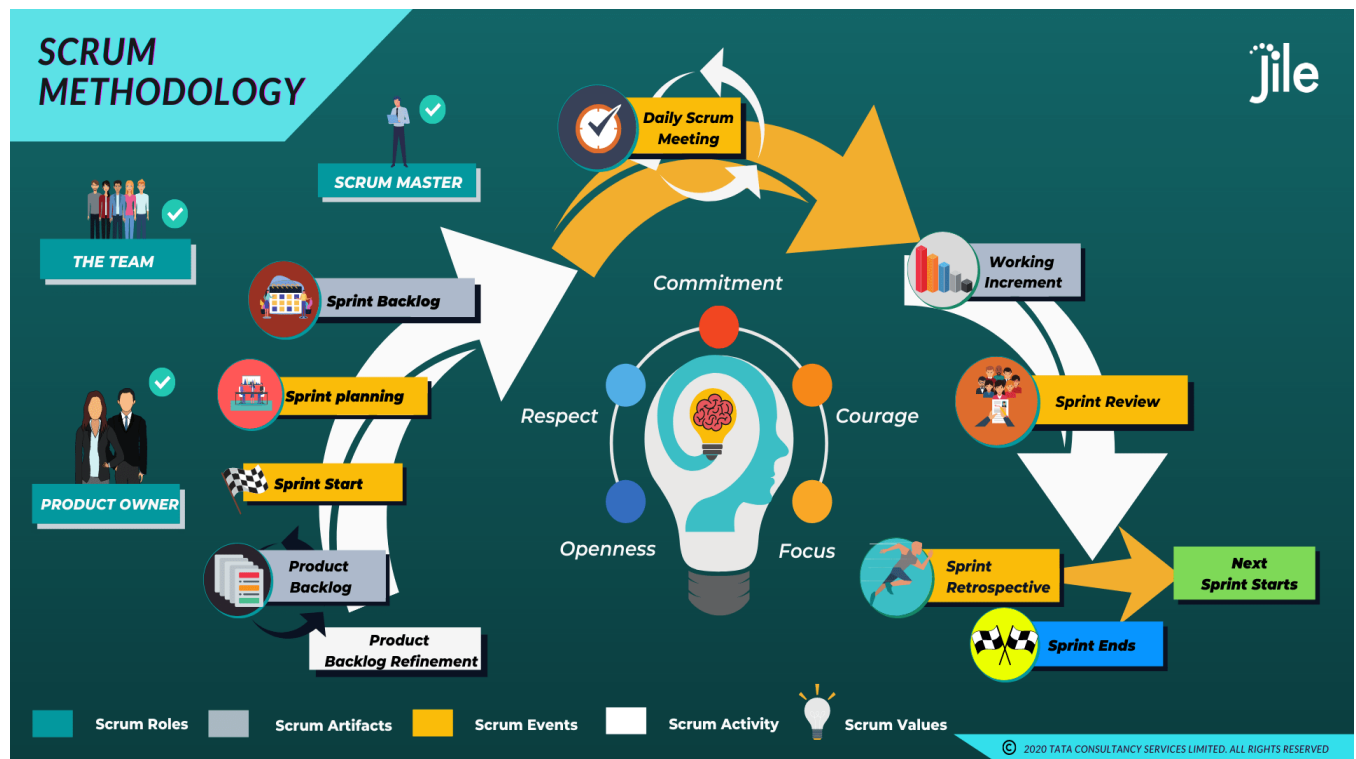
effective team collaboration. Ken Schwaber and Jeff Sutherland are co-creators of Scrum and continue to contribute significantly to the evolution of Scrum.

Scrum forms the base for many of the other frameworks and hence it is important for an agile practitioner to understand this methodology.

The Scrum Methodology is defined by:

- Scrum Roles
- Scrum Events
- Scrum Artifacts
- Scrum Values

Scrum has gained popularity in the software development community. Scrum Methodology has proven to scale across multiple teams of large organization with 900+ people. See how Jile supports Scrum Methodology.



What are Scrum Roles?

The scrum team is made up of three roles: A Product Owner, a Scrum Master and the Development Team.

1. Product Owner:

A Product Owner in a scrum team decides what needs to be built. This person has complete knowledge about the market and business needs, has a vision, and owns the return on investment (ROI) or the value delivered by the product.

Unlike traditional delivery, this person is a part of the team that delivers the product. Following are the key responsibilities of the Product Owner:

- Creates the vision
- Represents business, and is responsible for the ROI

- Cascades the vision to the teams
- Owns the backlog of features
- Prioritizes features by market value
- Is empowered to take decisions
- Negotiates with the team and business to deliver the right product at the right time

2. Scrum Master:

The Scrum Master is not a management title and cannot make decisions on behalf of the team. The Scrum Master's major responsibility is to ensure that scrum is understood and practiced by every team member in the true spirit.

The Scrum Master should understand the different skill sets of his or her team and group them by having the right sheep in the right flock. A Scrum Master should guide the team such that the team does not go astray and fall prey to excess time and energy.

Just like a shepherd, a Scrum Master must draw out quiet people during stand-up meetings or when planning poker sessions. Whenever the team loses focus or a team member goes astray, the Scrum Master aka the shepherd should bring the lost one back to the flock and guide appropriately.

The Scrum Master should not enforce agile practices on the team, but should do a 'Servant leadership' role. Scrum Master should lead by example and be a living demonstration of team assets and scrum values.

He or she should create an environment of safety for the team, and guide and facilitate team collaboration. He or she should refrain from solving problems or making decisions by guiding teams to do so.

To summarize, a Scrum Master:

- Is a servant leader - mentors and coaches the teams on scrum theory and practices, guides them on how they need to adapt to those, thereby realizing the benefits of scrum both at team level and organization level.
- Helps remove obstacles/impediments - supports the Development teams in removing the impediments by reaching out to the right people, thereby ensuring a smooth development progress without disrupting the team.
- Facilitates collaboration - enables interactions within the team as well as between the team and the Product Owner.
- Teaches scrum - to the team.
- Protects the teams - from external disruptions such as changes to stories in the current sprint.
- Is a change agent - in growing the organization to deliver early and often, and removing waste.

3. The Development team:

The Development team in scrum is the team that has all the skills necessary to execute the backlog items. This team is not a normal team but is committed, dedicated, and motivated to perform the best.

It is a self-organizing team that collaborates, shares their special skills and knowledge and are committed completely to fulfil the objective.

The following are the special characteristics of 'The Development team':

- Self-organizing - the Development team will be a self-managing group, who will decide on the tasks that they will work on incrementally. There is no 'Manager', who controls their work.
- Empowered - the team should commit to work, determine 'HOW' to deliver and decide on 'HOW MUCH' to deliver in one iteration.
- Cross-functional - the team does not segregate members as agile developers, testers or analysts and each member has the necessary skills to deliver the product increment.
- Small-sized - the Development team should ideally have 5 to 9 team members with skills sufficient to deliver the committed work. Smaller teams will not have the bandwidth to complete a considerable work and larger teams will increase the complexity.
- Co-located - the agile team is typically co-located to ensure effective collaboration.
- Committed - since the team is empowered to take decisions on the scope of work in a sprint, they are committed to delivery, should be transparent on the progress, and highlight the impediments early on.
- Dedicated - this team is focused and is 100% dedicated to product delivery.

Unlike traditional methodologies, where the commitment to deliver is made to business by the team that is not involved in the execution, in Agile, the team that does the work commits to how much work can be executed in a sprint.

The Development team decides how much work is to be done in a sprint, and commits to delivering a 'potentially shippable product increment (PSPI)', without sacrificing quality and speed. The team also makes continuous self-improvements.

What are Scrum Artifacts?

Scrum focuses more on a working software at the end of every sprint rather than comprehensive documentation. This does not imply that there is no documentation at all. The documentation is to facilitate collaboration and interactions, rather than tracking.

The progress is always measured through a working software. Scrum Documentation has three main artifacts namely: Product backlog, Sprint backlog, Increment.

1. Product backlog:

A product backlog is a dynamic list of functionalities the product might include, such that it provides value to users.

The Product Owner maintains this list and is responsible for creating, managing, and prioritizing the backlog by focusing on 'WHAT' brings the highest value to the users. These are a few unique characteristics of a product backlog:

It is dynamic in nature as it evolves based on changing market needs

Lists all the features and capabilities that will be taken up in iteration and delivered as a product increment

It is refined on a continuous basis. The Product Owner and Development team collaborate and update the details, estimate, and prioritize based on business value and size

2. Sprint backlog:

Sprint backlog is a subset of the entire product backlog that the scrum team plans to implement in one iteration or sprint.

During the sprint planning, the team selects items from the product backlog that they commit to complete in one sprint and thereby, create the sprint backlog. The Product Owner and Scrum Master should not provide inputs that may impact the team's decision. Sprint backlog has:

- Subset of product backlog items that the teams commit to implement in one sprint
- Items broken into smaller pieces of work as tasks
- A focus on 'HOW' the team does the work and delivers the value in one sprint
- A story or task board that is used by the teams to view backlog and what the individuals sign up for work after backlog prioritization
- Provision for the Development teams to track the sprint progress and check their alignment with sprint goals

3. Increment:

An increment is the work delivered at the end of every sprint. Typically, after every iteration there will be a Product Increment (PI) that delivers value and the final product will be a working software.

This increment is a sum of all the capabilities that were delivered in the previous sprints as a part of the PI. At the end of every sprint, the Product Owner decides whether to release the working product increment or wait until the next release.

What are Scrum Events?

All scrum activities are time-boxed and allow teams to inspect their current work and implement those learning's in future time-boxes.

Heart of Scrum - The Sprint

At the heart of scrum, is the 'Sprint'. The sprint is a time-boxed iteration, typically ranging from 1 to 4 weeks, at the end of which, a potentially shippable product increment is delivered by the Development team.

The sprint has the following characteristics:

- Does not exceed a maximum of one calendar month as this will increase the risk due to changes in requirements and therefore, may not provide the perceived business value at the end of the sprint
- Has a goal or 'definition of done' associated with every sprint that actually measures the success of the sprint
- Can be cancelled by the Product Owner, if the goal or the need for the sprint becomes obsolete due to changing market needs

Scrum advocates specific types of activities or meetings within a sprint to avoid the traditional formal meetings. These events and meetings are conducted at regular intervals and happen at specific periods of the sprint.

Typical Scrum Events are:

- Sprint planning
- Daily scrum
- Sprint review
- Sprint retrospective

Product backlog refinement (continuous activity throughout the sprint)

Product backlog refinement is a continuous activity throughout the sprint, where the Product Owner ensures that the product backlog is in order. The Product Owner performs the following tasks to ensure that the product backlog is relevant:

- Removes or demotes product backlog items that no longer seem important
- Adds or promotes product backlog items that become more important
- Splits product backlog items into smaller items or merges smaller ones into larger items and estimates those

1. Sprint planning (8 hours for a one- month Sprint & lesser duration for shorter Sprints)

Sprint planning meeting happens at the start of every sprint. This helps the Product Owner and Development teams to plan the product backlog items that will be taken up for implementation during the sprint. The Development team performs the following activities during this meeting:

- Considers and discusses product backlog items with the Product Owner
- Ensures a shared understanding about those items
- Selects a number of items that they estimate to complete
- Creates a sufficiently detailed plan to complete the selected items

There are two activities to be done to ensure that the above are achieved:

Part I: Define 'WHAT' work will be done

- Product Owner renders prioritized product backlog to the Development team

- The whole scrum team collaborates to understand the work
- The Development team is empowered and solely decides how much work is to be taken based on sprint capacity.
- The scrum team crafts a goal called the sprint goal as the essential focus of that sprint

Part II: Explain 'HOW' the work will get done

- Development team decides how to produce the next product increment that meets 'definition of done'
- Sufficient design and planning is conducted to complete the committed work
- Work to be done in initial days is split into small units of one day or even less
- Work to be done later are split whenever needed

2. Daily scrum (15 minutes)

Daily scrum is a 15-minute time-boxed event in which the team manages its daily activities. Daily Scrum is also called the daily stand-up meeting.

The scrum team meets every day, preferably at the same time and same place, so that it becomes a habit. During the meeting, each member answers three critical questions:

- What did I get done yesterday?
- What will I get done today?
- Are there any impediments blocking me?

It is essential that all the members of the development team are available for the daily stand-up meeting. The daily stand-up meeting is for the Development team to track progress towards the sprint goal, and they should participate enthusiastically to collaborate with each other.

The daily scrum also ensures that the impediments blocking the progress of the sprint are identified and resolved without further delay. Detailed problem solving does not happen during this meeting. Broadcasting individual updates to everyone in the team avoids unnecessary meetings.

This event enhances team communication and transparency, thereby enabling teams to be self-organized and take faster decisions.

3. Sprint review (4 hours for a one- month Sprint & lesser duration for shorter Sprints)

A sprint review is an event that happens at the end of every sprint, where the scrum team demonstrates the work that is done in the sprint to the stakeholders. The following happens during this meeting:

- A demo of the product increment showcasing the new features and underlying technology
- Feedback from the review provides input to the team to further discuss on refining the existing backlogs and plan for future sprints

- The Scrum Master facilitates this review meeting that is typically attended by all the stakeholders invited by the Product Owner
- Sprint review is essentially a way in which the team inspects and adapts to the next sprint and the overall product release

4. Sprint retrospective (3 - hours for a one - month Sprint & lesser duration for shorter Sprints):

During a sprint retrospective meeting, the scrum team inspects the previously completed sprint and identifies areas of improvement to be enacted for the upcoming sprints. This happens after every sprint and right after sprint review in which the whole scrum team participates.

During this meeting:

- The team introspects on what went well in terms of collaboration, planning, process, and tools
- They try to identify potential improvements that can be taken up in the next sprint to make the scrum processes more efficient by learning from previous shortfalls
- They decide on what could be done in the next sprint by taking into consideration the major improvements
- Scrum Master ensures that the teams improve their skills and knowledge during the scrum process so that they become more efficient in the next sprint
- The team focuses on improving their entire delivery cycle
- The three typical questions that the team answers are:
 - ❖ What shall we start doing?
 - ❖ What shall we stop doing?
 - ❖ What shall we keep doing?

All the above events in the scrum process framework enable teams to deliver a potentially shippable working software in short iterations. This also enables teams to capture feedback, inspect, and adapt for the next iteration.

What are Scrum Values?

Scrum also states five core values to which teams have to adhere. The core values are: Commitment, Courage, Focus, Openness, and Respect. These values should be imbibed and lived by the scrum team to ensure the fulfilment of scrum pillars of transparency, inspection, and adaptation. It builds trust among everyone.

Successful use of scrum depends on people becoming more proficient in these 5 values

- People personally commit to achieving the goals of the scrum team
- The scrum team members have the courage to do the right thing and work on tough problems
- Everyone focuses on the work of the sprint and the goals of the scrum team
- The scrum team and its stakeholders agree to be open to all the work and the challenges that they encounter while performing the work

- Scrum team members respect each other and consider everyone to be capable and independent

Benefits of Scrum methodology:

One of the main drawbacks of project management is the uncertainty of the results, so it is unthinkable to assure a project's success at its completion. Scrum ensures to be iterative, fast, and adaptive enough to deliver results as soon as possible in a project to decrease the gap between start and finish. Below are the reasons why scrum methodology is effective.

- Quick scrum deliverables - The projects practicing scrum framework are likely to deliver their outcomes (scrum deliverables) sooner in a productive manner.
- Easy to groom - Using the Scrum framework makes it flexible for everyone to refine in case any mistakes are identified mid-way and to fix those mistakes.
- Thrift usage - Time and money used effectively, whenever needed, thereby producing budget-friendly projects in a short period.
- Easy to cope - Scrum being a sub-group of agile can encompass changes in its project that are required by the client at ease with the help of its short sprint cycles and regular feedback.
- Lightweight process - This includes that the Scrum framework receives periodic updates of the progress done throughout the sprint through the events.
- Manageable units - However large a project may be, it could always be broken down into feasible items that can be realized in achievable sprints.
- Versatile - Grants access to any stage of development and allows maximum changeable actions within the scope of scrum methodology.
- Direct and clear - Each team member is aware in great clarity of each other's advancement made during the daily stand-up meetings.

Want to try a Scrum software for free?

Commonly, an individual scrum team uses Scrum tools such as sticky notes, a spreadsheet, or a whiteboard to manage the product backlog and measure the progress of the sprint backlogs in each sprint. This practice becomes complex once the team scales agile practices across the organization or to the enterprise level. The more teams use scrum within an organization that is geographically distributed, the more complicated it becomes with simple tools such as sticky notes, a spreadsheet, or a whiteboard.

Start your Agile journey by enabling software development and delivery teams to adopt Agile methods and practices such as Scrum and Kanban.

Jile is a single, Agile DevOps platform that provides multiple ways of working that can be configured based on the agile method that the teams wish to adopt. The teams can gradually evolve these methods as they scale up in their agile journey.

Jile provides six pre-configured WoW templates that are available out-of-the-box, including

- Scrum
- Kanban
- Enterprise Scaling Framework

- Disciplined Agile (DA)
- Large Scale Scrum (LeSS)
- Agile Portfolio

Each of these ways is pre-configured and can be customized based on organizational needs.

Jile enables companies of all sizes to grow and continuously evolve by making it easier to plan, develop, and deliver high-quality software that accelerates value delivery and supports innovation, reduces time to market, increases efficiency, and aligns software delivery with business initiatives.

1.4 Software Engineering | Extreme Programming (XP)

Extreme programming (XP) is one of the most important software development frameworks of Agile models. It is used to improve software quality and responsiveness to customer requirements. The extreme programming model recommends taking the best practices that have worked well in the past in program development projects to extreme levels.

Good practices need to be practiced in extreme programming: Some of the good practices that have been recognized in the extreme programming model and suggested to maximize their use are given below:

Code Review: Code review detects and corrects errors efficiently. It suggests pair programming as coding and reviewing of written code carried out by a pair of programmers who switch their works between them every hour.

Testing: Testing code helps to remove errors and improves its reliability. XP suggests test-driven development (TDD) to continually write and execute test cases. In the TDD approach, test cases are written even before any code is written.

Incremental development: Incremental development is very good because customer feedback is gained and based on this development team comes up with new increments every few days after each iteration.

Simplicity: Simplicity makes it easier to develop good quality code as well as to test and debug it.

Design: Good quality design is important to develop good quality software. So, everybody should design daily.

Integration testing: It helps to identify bugs at the interfaces of different functionalities. Extreme programming suggests that the developers should achieve continuous integration by building and performing integration testing several times a day.

Basic principles of Extreme programming: XP is based on the frequent iteration through which the developers implement User Stories. User stories are simple and informal statements of the customer about the functionalities needed. A User Story is a conventional description by the user of a feature of the required system. It does not mention finer details such as the different scenarios that can occur. Based on User stories, the project team proposes Metaphors. Metaphors are a common vision

of how the system would work. The development team may decide to build a Spike for some features. A Spike is a very simple program that is constructed to explore the suitability of a solution being proposed. It can be considered similar to a prototype. Some of the basic activities that are followed during software development by using the XP model are given below:

Coding: The concept of coding which is used in the XP model is slightly different from traditional coding. Here, the coding activity includes drawing diagrams (modeling) that will be transformed into code, scripting a web-based system, and choosing among several alternative solutions.

Testing: XP model gives high importance to testing and considers it to be the primary factor to develop fault-free software.

Listening: The developers need to carefully listen to the customers if they have to develop good quality software. Sometimes programmers may not have the depth knowledge of the system to be developed. So, the programmers should understand properly the functionality of the system and they have to listen to the customers.

Designing: Without a proper design, a system implementation becomes too complex and very difficult to understand the solution, thus making maintenance expensive. A good design results elimination of complex dependencies within a system. So, effective use of suitable design is emphasized.

Feedback: One of the most important aspects of the XP model is to gain feedback to understand the exact customer needs. Frequent contact with the customer makes the development effective.

Simplicity: The main principle of the XP model is to develop a simple system that will work efficiently in the present time, rather than trying to build something that would take time and may never be used. It focuses on some specific features that are immediately needed, rather than engaging time and effort on speculations of future requirements.

Pair Programming: XP encourages pair programming where two developers work together at the same workstation. This approach helps in knowledge sharing, reduces errors and improves code quality.

Continuous Integration: In XP, developers integrate their code into a shared repository several times a day. This helps to detect and resolve integration issues early on in the development process.

Refactoring: XP encourages refactoring, which is the process of restructuring existing code to make it more efficient and maintainable. Refactoring helps to keep the codebase clean, organized and easy to understand.

Collective Code Ownership: In XP, there is no individual ownership of code. Instead, the entire team is responsible for the codebase. This approach ensures that all team members have a sense of ownership and responsibility towards the code.

Planning Game: XP follows a planning game, where the customer and the development team collaborate to prioritize and plan development tasks. This approach helps to ensure that the team is working on the most important features and delivers value to the customer.

On-site Customer: XP requires an on-site customer who works closely with the development team throughout the project. This approach helps to ensure that the customer's needs are understood and met, and also facilitates communication and feedback.

Applications of Extreme Programming (XP): Some of the projects that are suitable to develop using the XP model are given below:

Small projects: XP model is very useful in small projects consisting of small teams as face-to-face meeting is easier to achieve.

Projects involving new technology or Research projects: This type of project face changing requirements rapidly and technical problems. So XP model is used to complete this type of project.

Web development projects: XP model is well-suited for web development projects as the development process is iterative and requires frequent testing to ensure the system meets the requirements.

Collaborative projects: XP model is useful for collaborative projects that require close collaboration between the development team and the customer.

Projects with tight deadlines: XP model can be used in projects that have a tight deadline, as it emphasizes simplicity and iterative development.

Projects with rapidly changing requirements: XP model is designed to handle rapidly changing requirements, making it suitable for projects where requirements may change frequently.

Projects where quality is a high priority: XP model places a strong emphasis on testing and quality assurance, making it a suitable approach for projects where quality is a high priority.

Extreme Programming (XP) is an Agile software development methodology that focuses on delivering high-quality software through frequent and continuous feedback, collaboration, and adaptation. XP emphasizes a close working relationship between the development team, the customer, and stakeholders, with an emphasis on rapid, iterative development and deployment.

Agile development approaches evolved in the 1990s as a reaction to documentation and bureaucracy based processes, particularly the waterfall approach. Agile approaches are based on some common principles, some of which are:

Working software is the key measure of progress in a project.

For progress in a project, therefore software should be developed and delivered rapidly in small increments.

Even late changes in the requirements should be entertained.

Face-to-face communication is preferred over documentation.

Continuous feedback and involvement of customer is necessary for developing good-quality software.

Simple design which evolves and improves with time is a better approach than doing an elaborate design up front for handling all possible scenarios.

The delivery dates are decided by empowered teams of talented individuals.

Extreme programming is one of the most popular and well-known approaches in the family of agile methods. an XP project starts with user stories which are short description of what scenarios the customers and users would like the system to support. Each story is written on a separate card, so they can be flexibly grouped.

XP, and other agile methods, are suitable for situations where the volume and space of requirements change is high and where requirement risks are considerable.

XP includes the following practices:

- **Continuous Integration:** Code is integrated and tested frequently, with all changes reviewed by the development team.
- **Test-Driven Development:** Tests are written before code is written, and the code is developed to pass those tests.
- **Pair Programming:** Developers work together in pairs to write code and review each other's work.
- **Continuous Feedback:** Feedback is obtained from customers and stakeholders through frequent demonstrations of working software.
- **Simplicity:** XP prioritizes simplicity in design and implementation, with the goal of reducing complexity and improving maintainability.
- **Collective Ownership:** All team members are responsible for the code, and anyone can make changes to any part of the codebase.
- **Coding Standards:** Coding standards are established and followed to ensure consistency and maintainability of the code.
- **Sustainable Pace:** The pace of work is maintained at a sustainable level, with regular breaks and opportunities for rest and rejuvenation.

XP is well-suited to projects with rapidly changing requirements, as it emphasizes flexibility and adaptability. It is also well-suited to projects with tight timelines, as it emphasizes rapid development and deployment.

- **Refactoring:** Code is regularly refactored to improve its design and maintainability, without changing its functionality.
- **Small Releases:** Software is released in small increments, allowing for frequent feedback and adjustments based on that feedback.
- **Customer Involvement:** Customers are actively involved in the development process, providing feedback and clarifying requirements.
- **On-Site Customer:** A representative from the customer's organization is present with the development team to provide continuous feedback and answer questions.
- **Short Iterations:** Work is broken down into short iterations, usually one to two weeks in length, to allow for rapid development and frequent feedback.
- **Planning Game:** The team and customer work together to plan and prioritize the work for each iteration, with the goal of delivering the most valuable features first.
- **Metaphor:** A shared metaphor is used to guide the design and implementation of the system.

- **Coding Standards:** Coding standards are established and followed to ensure consistency and maintainability of the code.

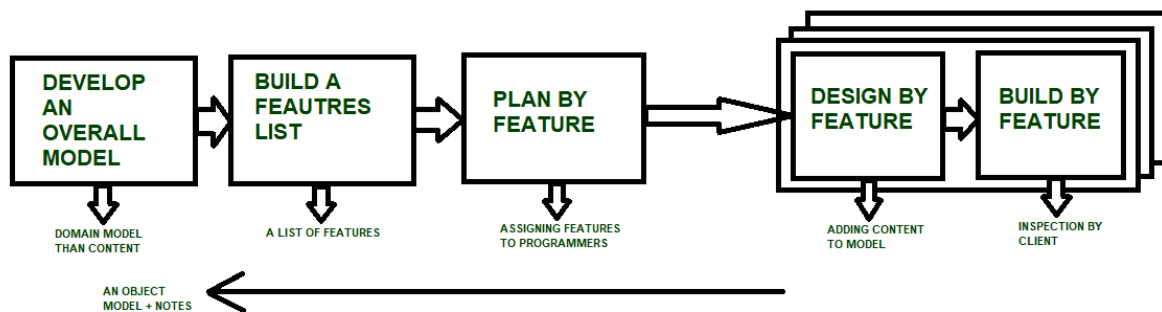
Advantages of Extreme Programming (XP):

- Slipped schedules – Timely delivery is ensured through slipping timetables and doable development cycles.
- Misunderstanding the business and/or domain – Constant contact and explanations are ensured by including the client on the team.
- Canceled projects – Focusing on ongoing customer engagement guarantees open communication with the consumer and prompt problem-solving.
- Staff turnover – Teamwork that is focused on cooperation provides excitement and goodwill. Team spirit is fostered by multidisciplinary cohesion.
- Costs incurred in changes – Extensive and continuing testing ensures that the modifications do not impair the functioning of the system. A functioning system always guarantees that there is enough time to accommodate changes without impairing ongoing operations.
- Business changes – Changes are accepted at any moment since they are seen to be inevitable.

Production and post-delivery defects: Emphasis is on – the unit tests to find and repair bugs as soon as possible.

1.5 FDD Full Form

FDD stands for Feature-Driven Development. It is an agile iterative and incremental model that focuses on progressing the features of the developing software. The main motive of feature-driven development is to provide timely updated and working software to the client. In FDD, reporting and progress tracking is necessary at all levels.



Characteristics of FDD

- **Short iterative:** FDD lifecycle works in simple and short iterations to efficiently finish the work on time and gives good pace for large projects.
- **Customer focused:** This agile practice is totally based on inspection of each feature by client and then pushed to main build code.

- **Structured and feature focused:** Initial activities in lifecycle builds the domain model and features list in the beginning of timeline and more than 70% of efforts are given to last 2 activities.
- **Frequent releases:** Feature-driven development provides continuous releases of features in the software and retaining continuous success of the project.

Advantages of FDD

- Reporting at all levels leads to easier progress tracking.
- FDD provides continuous success for larger size of teams and projects.
- Reduction in risks is observed as whole model and design is build in smaller segments.
- FDD provides greater accuracy in cost estimation of the project due to feature segmentation.

Disadvantages of FDD

- This agile practice is not good for smaller projects.
- There is high dependency on lead programmers, designers and mentors.
- There is lack of documentation which can create an issue afterwards.

1.6 Kanban - Lean and Agile

In Agile Kanban, the user stories are broken into tasks and Kanban cards are used to track the tasks on the Kanban board. Agile Kanban has a concept of iteration that is not present in Kanban. Further, no processes are considered.

Kanban in Value Stream

Kanban is defined to be executed in value stream with focus on delivery of value. Kanban in software development can be visualized as the features flowing across the value stream. All the Kanban characteristics (Refer Chapter - Characteristics of Kanban in this Tutorial) are met in the Kanban approach for software development.

Feature Kanban Board

Feature Kanban Board is used to track the Feature Driven Development with Kanban Approach. Each Feature is assigned to a particular release. The columns in the Kanban board represent releases. Hence, each column contains all the features assigned to the release represented by it.

Each feature is broken into stories. Each release is broken into iterations. The iteration is executed in an Agile Development approach. This can be treated as a sub-stream in the value stream, with the stories to be completed within that iteration assigned to it.

Agile Kanban in Sub-stream

Agile Kanban approach is followed within each sub-stream that is implemented as an iteration. Each story is broken into tasks in the iteration. Task Kanban board is used to track the status and progress of the story development tasks. The current status of each task is known by displaying the cards in separate columns on the board. The columns are labeled as To Do, Doing, and Done. Each task moves from To Do to Doing and then to Done.

- Continuous Delivery

- Continuous delivery to the customer is ensured with features tracked on feature Kanban board and stories representing features tracked on task Kanban board.

Delivery through a release is accomplished by –

- Continuous tracking
- Constant communication with the customer
- Adjusting development plan as required
- Focusing on delivery of value to the customer
- Agile development as well as Kanban maintain team collaboration. This, in turn helps in identifying and resolving Bottlenecks immediately as required by Kanban. This results in accomplishment of all the needed tasks within the iteration to deliver quality product, which meets customer expectations.

Continuous Process Improvement

Kanban supports process improvements to enhance the delivery approach continuously.

Consider a requirement that is a change or addition to the product. In such a case, Kanban cards can be used to visualize the requirement passing through the processes of analysis, design, development, product integration and testing. This is different from the Waterfall approach in the sense that it does not require completion of one process for all the requirements to flow to the next process in the sequence.

Such an implementation of Kanban in product maintenance allows maintainability, reliability and integrity of the product. The required process improvements are gathered at regular intervals and implemented on a continuous basis.

1.7 Overview of DevOps

Report linker also predicts that the DevOps market size would grow up to US\$10.3 billion by 2023, at a compound annual rise rate of 24.70 percent during the forecast period. Being in the technology field, you must be aware of this buzzword, DevOps, taking up the IT world by storm. This tutorial will take you through the main concepts of DevOps, along with its important tools and use cases.

What exactly is DevOps?

Don't get scared with the term 'DevOps.' It is nothing but the practice or methodology of making 'Developers' and the 'Operations' team work together.



Why DevOps?

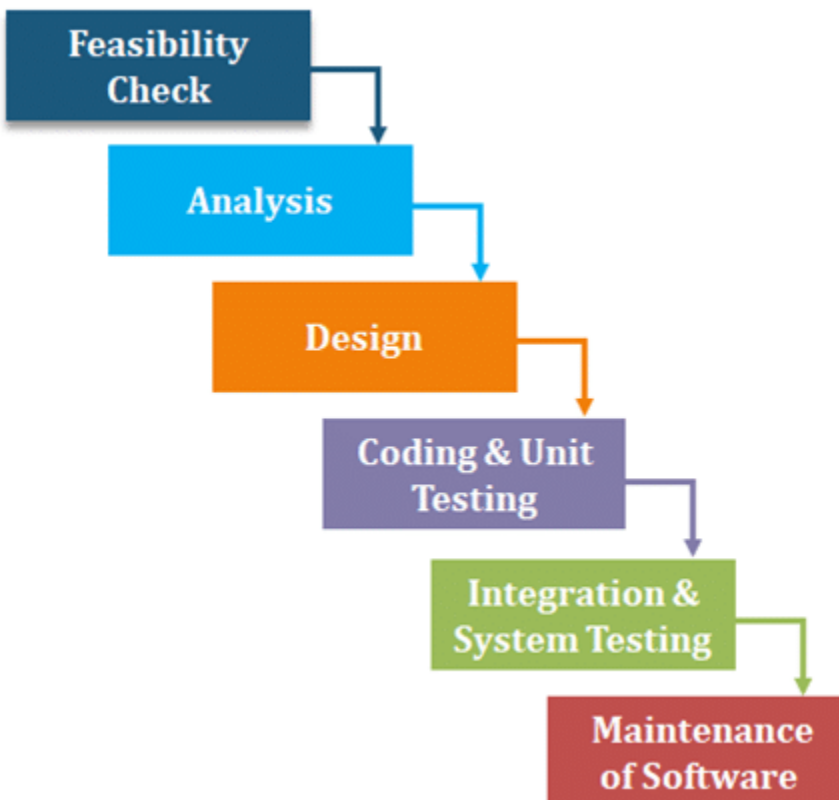
Before understanding the concepts and methodology of DevOps, we need to understand why do we even need DevOps?

Why DevOps? Why not other methods?

Before DevOps came into the picture, the Waterfall model was the earliest SDLC approach that was used for software development. This method, which was used for illustrating SDLC in a sequential flow, was considered to be reliable at first.

The Workflow of the Waterfall Method

Let's consider, we're developing software using the Waterfall method. Below are the steps that will be included in the SDLC if we're using this method:



Feasibility check: The feasibility phase is used for determining whether a particular approach/technique will be feasible enough for developing the software.

Analysis of the requirements: In this phase, we need to analyze all system and software requirements from customers' points of view and gather information about these requirements. The requirements will then be captured in the software requirement specification (SRS) document to avoid the incompleteness of the product.

Design: The goal of this phase is to transform the requirements listed in the SRS document into an ordered structure that is appropriate for their implementation in programming.

Learn the Azure DevOps interview Questions to have a better aspect of the industry-level questions.

Coding and unit testing: The design that is created in the previous stage is supposed to get converted into the source code in this stage, and then every design module is coded and checked individually.

Integration and system testing: After the design of each module has been coded, the integration of these modules is carried out appropriately. Then, these integrated modules are tested individually. After this, acceptance testing is carried out in which the product is delivered to and tested by the customer for checking whether to accept it or reject it.

Maintenance of the software: Maintenance is that phase of the software development life cycle where 60 percent of the entire effort is spent. Several maintenance operations are performed in this phase such as corrective maintenance, perfective maintenance, and adaptive maintenance, where error corrections and functionality enhancement, along with trying the software on new environments and operating systems, are done.

Advantages

- This method is easy and simple to use
- Easy to manage due to its rigidity
- Each phase has a review process making it less vulnerable to errors
- The one-at-a-time process phases do not overlap each other
- Reliable for small projects

Disadvantages

- While the application is in the testing stage, it is really difficult to go back and make changes relating to any issue that happened in the previous steps due to miscommunication or lack of knowledge
- It is a risky process as it is difficult to diagnose and to provide feedback
- Its main focus is to help internal teams work efficiently. It excludes end-users/clients, due to which the majority of people do not trust this methodology
- There will be delays in the testing process because this method insists teams to wait until the process reaches its 4th or 6th stage

- Because of all these disadvantages, organizations wanted a much efficient model to carry out SDLC. Hence came the Agile method that changed the scenario.

Agile Software Development

Agile involves an incremental approach like the Waterfall model but with an iterative perspective, along with focusing on customer feedback, incorporating small rapid changes, and speeding up releases. It basically breaks the product into smaller divisions and finally integrates them for the testing process.

Advantages

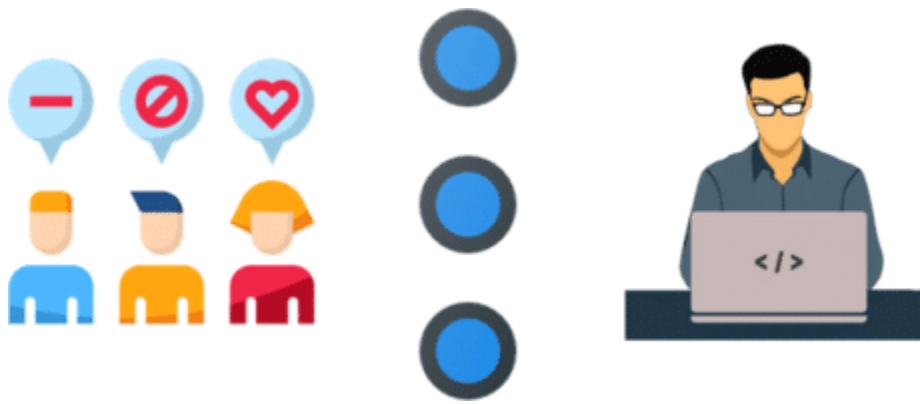
- The Agile methodology considers customer feedback throughout the project, which gives enough time to the team for making decisions
- It welcomes making changes but at great expense
- It has the ability to scale
- There is continuous attention to technical excellence and good designs
- This method prioritizes and schedules the most valuable features for implementation, decreasing the risk of having unusable resources
- Small and dedicated teams are involved with a high degree of involvement and coordination

Disadvantages

- There is less predictability in Agile
- It requires more time and commitment from every stakeholder. Testers, developers, and customers must interact with each other constantly and should agree to each other's decisions in order to get the task done, and hence Agile is time-consuming
- Limited documentation often comes as a problem. In the case of fallbacks, there are very less detailed documents so as to cross-check
- The Agile model requires minimal planning at the beginning that makes it easier to develop the project quickly, but there is never a finite end. Due to unexpected functionalities, a clear vision of the project is not available, and mostly the stakeholders are not sure of what their final product would look like

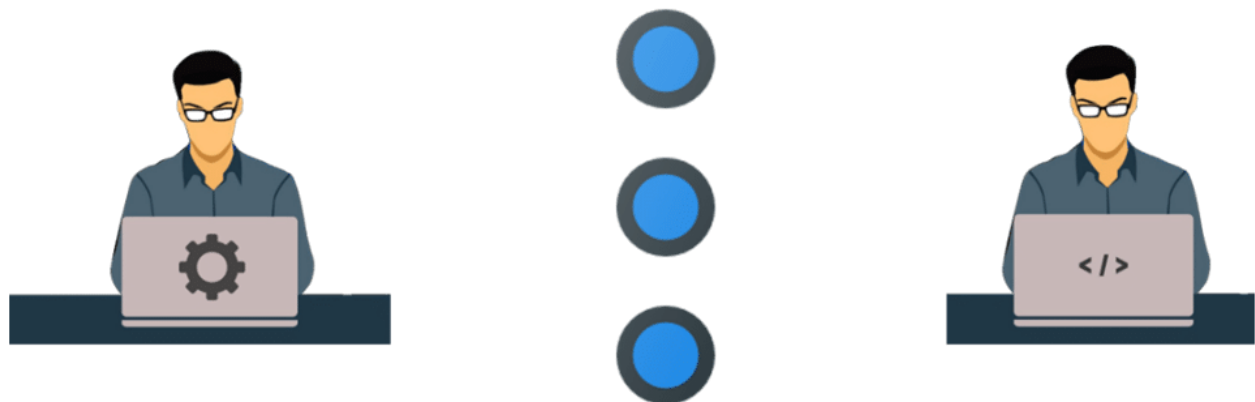
There were many disadvantages to the Agile model as well:

In the case of the Waterfall model, there was a gap between customers' software requirements and the developers, which was overcome by Agile



Gaps Between Customers and Developers

While in the case of the Agile method, there was still a gap between the development and operations folks



Look into the Monitoring DevOps Tools blog provided by Intellipaat.

Moving to the next section in this DevOps tutorial, let's check out the major differences between Agile and DevOps.

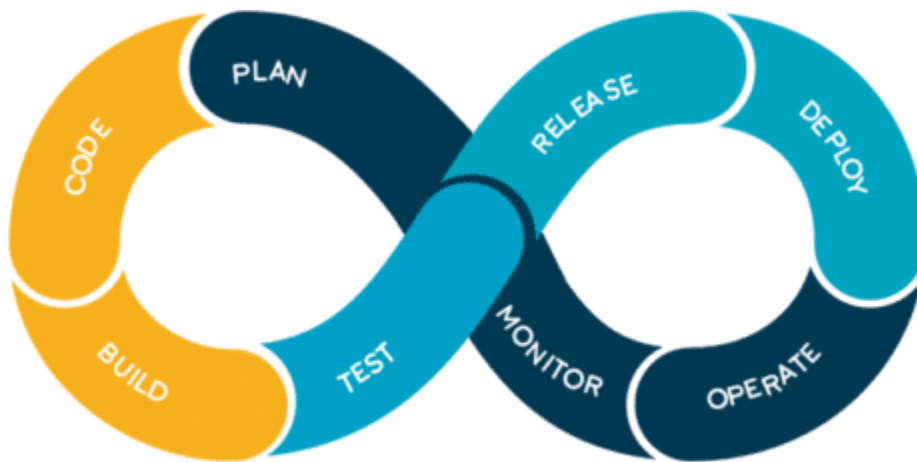
Differences Between Agile and DevOps

Agile	DevOps
Agile majorly focuses on collaboration, customer feedback, and small rapid changes	DevOps brings development and operations teams together
It does not focus on automation	It focuses majorly on automation to increase efficiency while deployment
The development process is inherent for Agile, making it less focused on testing and implementation processes	DevOps focuses on all development, testing, and implementation phases with equal importance
It overcomes the gap between customers and developers	It overcomes the gap between the development and operations folks

DevOps Lifecycle

DevOps focuses on bringing all the development, operations, and IT infrastructure guys, including Developers, Testers, System Admins, and QAs, under one roof. Hence, all these people together are called DevOps Engineers.

DevOps Engineers share the end-to-end responsibility of gathering information, setting up the infrastructure, developing, testing, deploying, continuously monitoring, and fetching feedback from end-users. This process of developing, testing, deploying, and monitoring keeps on repeating for better results. You can actually figure it all out from the DevOps diagram illustrated below:



- **Code:** The first step in the DevOps life cycle is coding, where developers build the code on any platform
- **Build:** Developers build the version of their program in any extension depending upon the language they are using
- **Test:** For DevOps to be successful, the testing process must be automated using any DevOps automation tool like Selenium
- **Release:** A process for managing, planning, scheduling, and controlling the build in different environments after testing and before deployment
- **Deploy:** This phase gets all artifacts/code files of the application ready and deploys/executes them on the server
- **Operate:** The application is run after its deployment, where clients use it in real-world scenarios.
- **Monitor:** This phase helps in providing crucial information that basically helps ensure service uptime and optimal performance
- **Plan:** The planning stage gathers information from the monitoring stage and, as per feedback, implements the changes for better performance

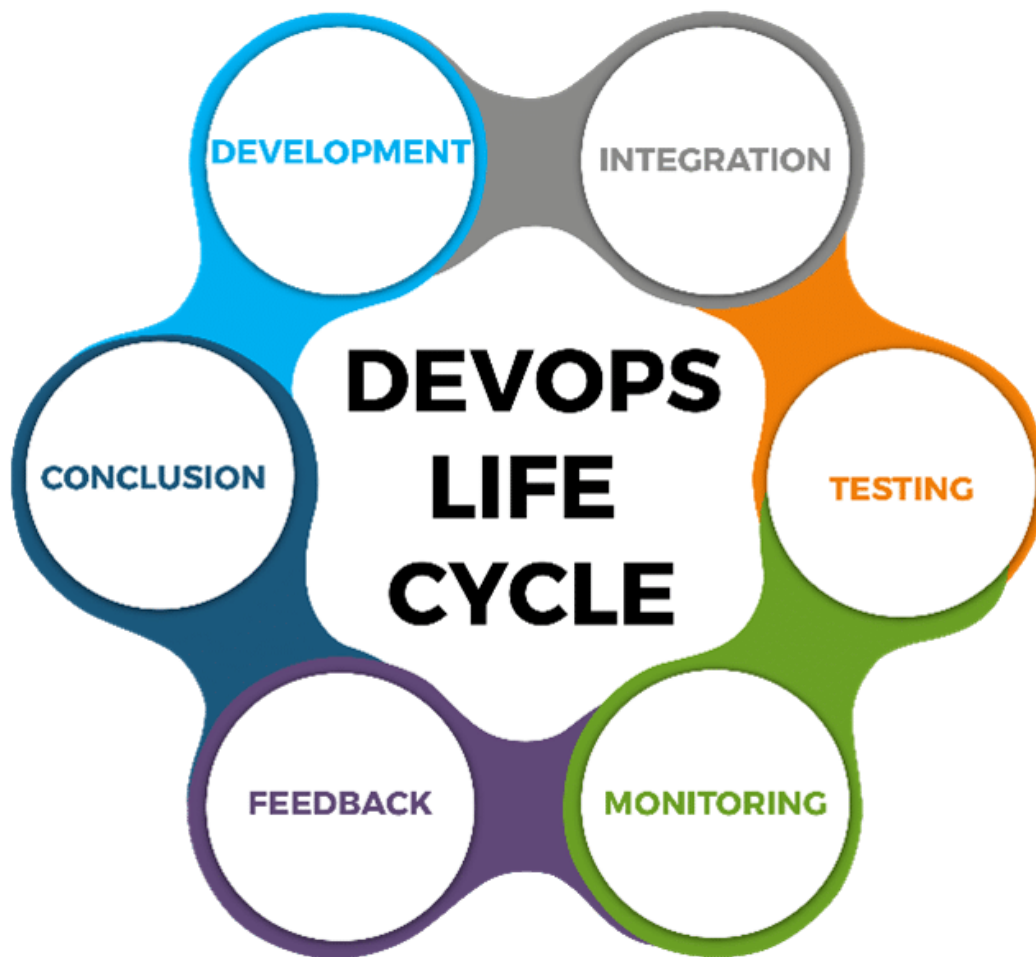
Different Lifecycle Stages

Now, let's discuss the different stages in the DevOps life cycle that contributes to the consistent software development life cycle (SDLC):

- Continuous Development
- Continuous Integration

- Continuous Testing
- Continuous Monitoring
- Virtualization and Containerization

These stages are basically the aspects of achieving the DevOps goal.



Continuous Development

In the Waterfall model, our software product gets broken into multiple pieces or sub-parts for making the development cycles shorter, but in this stage of DevOps, the software is getting developed continuously.

Tools used: As we code and build in this stage, we can use GIT to maintain different versions of the code. To build/package the code into an executable file, we can use a reliable tool, namely, Maven.

GIT Cheat Sheet by Intellipaat is one of the most visited and downloaded. Check out.

Continuous Integration

In this stage, if our code is supporting new functionality, it is integrated with the existing code continuously. As the continuous development keeps on, the existing code needs to be integrated with the latest one ‘continuously,’ and the changed code

should ensure that there are no errors in the current environment for it to work smoothly.

Tools used: Jenkins is the tool that is used for continuous integration. Here, we can pull the latest code from the GIT repository, of which we can produce the build and deploy it on the test or the production server.

Continuous Testing

In the continuous testing stage, our developed software is getting tested continuously to detect bugs using several automation tools.

Tools used: For the QA/Testing purpose, we can use many automated tools, and the tool used widely for automation testing is Selenium as it lets QAs test the codes in parallel to ensure that there is no error, incompetencies, or flaws in the software.

Continuous Monitoring

It is a very crucial part of the DevOps life cycle where it provides important information that helps us ensure service uptime and optimal performance. The operations team gets results from reliable monitoring tools to detect and fix the bugs/flaws in the application.

Tools used: Several tools such as Nagios, Splunk, ELK Stack, and Sensu are used for monitoring the application. They help us monitor our applications and servers closely to check their health and whether they are operating actively. Any major issue detected by these tools is forwarded to the development team to fix in the continuous development phase.

DevOps Tools



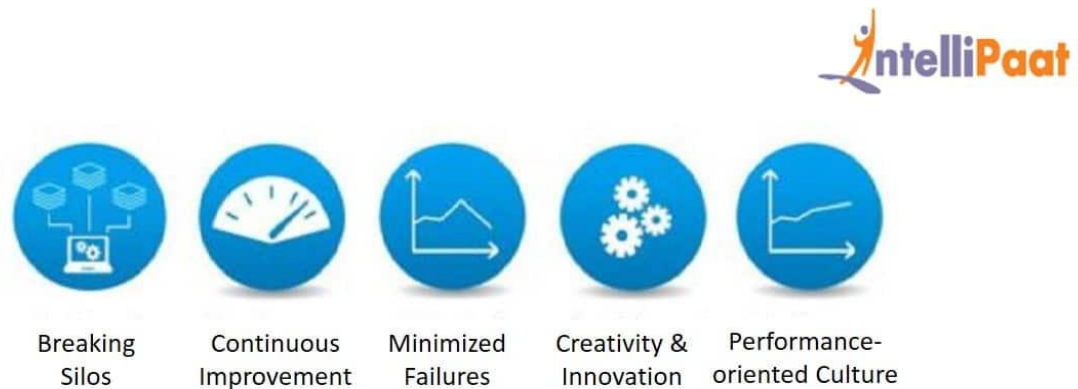
- Puppet: Puppet is one of the widely-used DevOps tools. It allows delivering and releasing technology changes quickly and frequently. It has features of versioning, automated testing, and continuous delivery.

- **Docker:** Docker is a high-end DevOps tool that allows building, shipping, and running distributed applications on multiple systems. It helps assemble the applications quickly and is typically suitable for container management.
- **Jenkins:** Jenkins is one of the most popular DevOps tools that allow monitoring of the execution of repeated jobs. Apart from this, Jenkins lets us integrate the changes and access the results easily and quickly.
- **Ansible:** This tool helps automate the entire life cycle of an application, and manages complicated deployments, and enhances productivity.
- **Nagios:** This DevOps tool helps monitor the IT infrastructure. It is capable of determining errors and rectifying them with the help of the standard network, server, and log monitoring systems.

DevOps Benefits

After being successfully implemented in SDLC, now DevOps is considered the key to speeding up various cloud platforms. Its all-rounder performance has attracted aspirants to build a career in this domain, and hence having sound knowledge is imperative for them.

DevOps is a contemporary approach that lets companies utilize numerous benefits. Some of the major DevOps benefits are as follows:



DevOps Use Case in Netflix

DevOps We all know about Netflix, the world's leading media-subscription provider that streams various TV shows and movies on our favorite smart devices, delivering the best experience anywhere at any time to more than 75 million global customers.

Let's understand how Netflix uses DevOps to provide its customers with the best and smooth video streaming experience. Netflix uses Spinnaker's continuous delivery platform for the continuous delivery of its application. Before reaching Spinnaker, there are a number of steps that are supposed to take place.

Let's understand this from the diagram below:

Code Check-in	Continuous Integration	Bake	Deployment	Deployment	Post-Deploy Test	Hotfix	Canary	Live
---------------	------------------------	------	------------	------------	------------------	--------	--------	------

SPINNAKER

GIT, NEBULA, JENKINS, BAKERY



Netflix uses Nebula for the build; it considers Nebula as the best build tool for Java applications. It is a collection of Gradle plugins meant for Netflix engineers to eliminate boilerplate build logic and provide sane conventions.

```

1  apply plugin: 'nebula'
2  apply plugin: 'war'
3  apply plugin: 'netflix.ospackage-tomcat'
4  apply plugin: 'nebula.dependency-lock'
5
6  dependencies {
7      compile 'netflix:base-server:latest.release'
8      compile 'javax.ws.rs:jsr311-api:1.1.1'
9      provided 'javax.servlet:javax.servlet-api:3.1.0'
10
11     testCompile 'junit:junit-dep:4.10'
12     testCompile 'org.mockito:mockito-all:1.9.5'
13 }
14
15 ospackage {
16     requires('apache-tomcat8')
17 }

```

A Simple Java Application Build.gradle File

Further, the code is tested locally using Nebula. The changes, if any, are committed to their central Git repository. Also, Netflix migrates its monolith application to the cloud-based microservices in AWS. The microservice architecture allows teams at Netflix to be loosely coupled, building and pushing changes at a speed they are comfortable with.

A Jenkins job is created that helps execute Nebula, which further builds, tests, and packages the application for further deployment. The build is further 'baked' into an Amazon Machine Image (AMI). For generating AMIs from the source, Netflix

creates a bakery, and the bakery exposes an API that facilitates the creation of AMIs globally.

Once the baking is complete, Spinnaker comes into the picture that helps in deployment by making the resultant AMIs available for the tens, hundreds, and thousands of instances.

After this continuous integration, deployment, and final availability, the application goes live.

DevOps Engineers are among the highest paid professionals in the technology domain! Join DevOps Training in Hyderabad!

Since we thoroughly understand what DevOps is and how it works, now, we need to know the various DevOps Engineer skills one should possess.

Roles and Responsibilities of a DevOps Engineer

DevOps is more of a culture that is being incorporated by the giants in the IT world currently. DevOps, when practiced the right way keeping certain roles and responsibilities in mind, helps overcome the gap between development and operations teams.

The roles and responsibilities of DevOps Engineers are as follows:



- Project planning and management: In addition to monitoring software and regulating and updating tools, DevOps Engineers need to have expertise in keeping track of the costs, benefits, risks, and much more of various DevOps projects.
- Design, development, and deployment: DevOps Engineers are required to design, develop, and deploy automated modules for smooth functioning within the production environment, by utilizing risk-management techniques, tests, etc.
- Communication and support: DevOps Engineers should have exceptional communication skills that come in handy when they need to work and coordinate with different departments and provide support.
- Technical skills: Some basic technical experience and familiarity with configuration tools are a must.
- Interpersonal skills: Since DevOps Engineers are in constant interaction with other departments in the organization, they should be approachable, organized, and foresighted team players with an ability to multitask.
- Troubleshooting: Last but not least, one of the major responsibilities of DevOps Engineers is to troubleshoot and come up with apt solutions for various errors to benefit the firm with speed and efficiency.

The main aim behind adopting all these roles and responsibilities is to be able to perform coding, scripting, and process re-engineering.